# From Stochastic to Semantic: Advanced Attribute-Guided Compiler Testing

JIANGCHANG WU, YIBIAO YANG, MAOLIN SUN, QINGYANG LI, KANG CHEN, LEI XU, and YUMING ZHOU, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China

Compiler testing is critically important, as compilers serve as the foundational infrastructure in system software development. A comprehensive exploration of the compilation space is essential for uncovering bugs in compilers. Existing methods primarily involve the utilization of various compilation options alongside test programs as inputs for stress-testing compilers. However, these compilation options are typically applied uniformly across all program elements—such as functions and variables, by default, limiting the ability to thoroughly explore the compilation space. In programming languages like C and C++, attributes such as the `__attribute__((always_inline))` directive provide a mechanism for programmers to specify additional information for specific code elements to the compiler. These attributes allow for precise control over the compilation process, such as enforcing constraints and customizing optimization passes for particular elements. This flexibility in specifying attributes offers opportunities to investigate previously unexamined areas within compilers. Unfortunately, few studies have leveraged attributes for compiler testing. To this end, we propose Atlas, an attribute-guided approach that strategically inserts attributes into test programs to facilitate a more thorough exploration of the compilation space. Our key insight is that attributes specified for individual program elements can provide a more flexible means of exploring the compilation space. Our extensive experiments on GCC and LLVM demonstrate the superiority of Atlas over baseline testing techniques that do not employ attributes, particularly in terms of bug detection and code coverage. Furthermore, Atlas has led to the discovery of 97 unique bugs in GCC and LLVM, 73 of which have already been confirmed or fixed, showcasing its practical utility.

CCS Concepts: • **Software and its engineering → Software testing and debugging**;

Additional Key Words and Phrases: Compiler Testing, Attribute-guided, Compilation Space Exploration, Optimization, Large Language Model

This work was supported in part by the National Natural Science Foundation of China (Grants 624B2067, 62572226, and 62272214), the Jiangsu Natural Science Foundation (Grant BK20231402), and the Collaborative Innovation Center of Novel Software Technology and Industrialization.

Authors' Contact Information: Jiangchang Wu, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China; e-mail: jiangchangwu@smail.nju.edu.cn; Yibiao Yang (corresponding author), State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China; e-mail: yangyibiao@nju.edu.cn; Maolin Sun, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China; e-mail: merlin@smail.nju.edu.cn; Qingyang Li, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China; e-mail: liqingyang@smail.nju.edu.cn; Kang Chen, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China; e-mail: k.chen@smail.nju.edu.cn; Lei Xu, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China; e-mail: xlei@nju.edu.cn; Yuming Zhou, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China; e-mail: zhouyuming@nju.edu.cn.

This work is licensed under Creative Commons Attribution International 4.0.

© 2026 Copyright held by the owner/author(s).
ACM 1557-7392/2026/10-ART337
https://doi.org/10.1145/3793554

**ACM Reference format:**
Jiangchang Wu, Yibiao Yang, Maolin Sun, Qingyang Li, Kang Chen, Lei Xu, and Yuming Zhou. 2026. From Stochastic to Semantic: Advanced Attribute-Guided Compiler Testing. *ACM Trans. Softw. Eng. Methodol.* 35, 10, Article 337 (October 2026), 38 pages.
https://doi.org/10.1145/3793554

## 1 Introduction

Compilers are fundamental to the development of system software, as nearly all system software relies on them for construction [3]. However, the complexity and extensive scope of compilers make them prone to errors. Bugs hidden within compilers can have far-reaching consequences, affecting system correctness, security, reliability, performance, and maintenance. Therefore, ensuring the reliability and correctness of compilers is of critical importance.

To ensure the quality of compilers, various compiler testing methods have been proposed [5]. Csmith [65], a notable tool in this domain, randomly generates diverse test programs based on predefined rules or grammar. More recently, YARPGen [37] and its successor YARPGen v.2 [38] have been proposed, focusing on generating test programs specific to the compiler's loop optimization strategies. Both Csmith and YARPGen are capable of generating a large number of test programs to stress test compilers. During the testing process, different combinations of compilation options are employed which aim to explore the compilation space, thereby exposing potential bugs in compilers. **Equivalence Modulo Inputs (EMI)** [29], a representative metamorphic testing technique [9], mutates existing programs by removing or inserting random code while preserving the original input/output behaviors. EMI also applies various compilation options to compile the mutated programs which also intend to trigger unexplored behaviors of compilers. This line of research has yielded significant results, uncovering a considerable number of bugs in mature and well-established compilers [30, 54].

However, existing compiler testing methods primarily depend on the use of different compilation options to explore compilation space. These options are typically applied to the entire program, which limits their capabilities to explore the deeper states of compilers. In practice, *attributes* in programming languages such as C and C++ provide a valuable mechanism for programmers to specify additional information to the compiler to enforce constraints, optimize specific sections of code, or facilitate targeted code generation. The additional information can include the intended use and behavior of variables, functions, and types [13–15], which can influence various stages of the compilation process, from optimization to code generation, and even runtime behavior. For instance, the `noinline` attribute instructs the compiler not to inline a function during optimization [13]. The `aligned` attribute specifies the necessary alignment for data structures during code generation [15]. Similarly, the `noreturn` attribute indicates that a function does not return, allowing the compiler to optimize control flow and potentially eliminate unnecessary cleanup code [13].

*Approach.* In this article, we propose Atlas,[1] an attribute-guided comprehensive compiler testing technique. The key insight underlying Atlas is that attributes provide flexible control over the compilation process for individual elements within test programs, thereby facilitating a more thorough exploration of compiler behavior. Specifically, for a given test program, Atlas first identifies the program elements, i.e., the functions, variables, and statements, where potential attributes can be specified. Once these elements are identified, Atlas employs two complementary strategies to generate mutated test programs:

[1] Atlas: *At*tribute-Guided Compi*la*tion *S*pace Exploration.

— Our **Stochastic Strategy (SS)** randomly inserts appropriate attributes for those elements, resulting in mutated test programs with the inserted attributes.
— Our **LLM-Guided Semantic-Aware Strategy (LGS)** leverages a **Large Language Model (LLM)** to first comprehend the functionality and context of a code fragment. Based on this semantic understanding, it then recommends highly relevant attributes for insertion.

These mutated test programs are then fed to compilers for subsequent testing. By strategically combining various compilation options with the attributes inserted programs, Atlas can thoroughly explore the functionalities of the compiler, thereby improving the effectiveness in exposing compiler faults.

To evaluate the performance of Atlas, we conduct an extensive study based on GCC and LLVM. In the study, we compared Atlas with those baseline techniques without inserting any attributes. Our experimental results show that Atlas outperforms these methods in terms of the number of bugs found within the same testing period. In particular, Atlas can detect 109.1% more bugs than the baseline approaches on average. Moreover, Atlas achieves higher code coverage of GCC and LLVM triggered by the mutated test programs than the baseline approaches. Furthermore, we applied Atlas to test the trunk version of both GCC and LLVM. In total, Atlas has successfully detected 97 unique bugs, 73 of which have been confirmed or fixed by developers, demonstrating the effectiveness of Atlas.

*Contributions.* We make the following major contributions:

— *Novelty:* We introduce an attribute-guided **Compilation Space Exploration (CSE)** approach for comprehensive compiler testing. Specifically, we propose to strategically inserting appropriate attributes into program elements to systematically explore the compilation space for testing compilers.
— *Prototype:* We have developed a prototype implementation of our proposed approach, named Atlas, which leverages various attributes to more effectively explore the compilation space of compilers.
— *Usefulness:* We conduct an extensive study to evaluate Atlas on two of the most widely used C compilers (i.e., GCC and LLVM), demonstrating its effectiveness. In particular, Atlas has led to 97 unique bugs for GCC and LLVM, 73 of which have been confirmed or fixed by developers.

*Significance.* This study extends and substantially differs from our earlier work [61] as follows:

— *LLM-Guided Semantic-Aware Attribute Generation.* While our initial SS is effective for broad exploration, it is also limited by its random nature. First, the strategy lacks specificity, making it difficult to uncover bugs that are related to specific semantic contexts of the code. Second, this randomness often leads to the insertion of attributes that are syntactically valid but semantically incompatible with the code context, resulting in a large number of uncompilable test cases. To this end, we introduce a novel testing strategy, the LGS, as a powerful complement to the original stochastic approach. This strategy moves beyond random attribute insertion by leveraging the code comprehension capabilities of LLMs. For each program element, Atlas extracts a complete semantic code snippet and queries an LLM. Prompted to act as a compiler testing expert, the LLM first develops a semantic understanding of the code's functionality and context and then leverages this insight to recommend syntactically correct and semantically relevant attributes. Crucially, it also provides the corresponding compilation options to activate the attribute. This enhancement enables a deeper and more targeted exploration of the compilation space.

— *Discovery and Confirmation of New Compiler Bugs.* The practical effectiveness of our new LGS is demonstrated by the discovery of 24 new, previously unknown bugs in mainstream compilers. This includes 20 bugs in GCC and 4 in LLVM, 15 of these 24 bugs have already been confirmed by the respective compiler developers.
— *Comprehensive Evaluation of Stochastic and LLM-Guided Strategies.* We have substantially expanded our empirical evaluation to analyze and contrast the performance of the new LGS against the original SS and other baselines. Our analysis includes a comparison of the unique bugs found, the bug discovery rates, and code coverage. The results reveal a valuable tradeoff: while the SS finds more bugs, the LGS achieves higher code coverage and a better success rate of generating compilable programs, confirming the value of both complementary approaches. To complete this analysis, our discussion on Threats to Validity has also been extended to address the challenges inherent to the LLM-guided strategy, such as its dependency on a specific model and the impact of prompt engineering.

*Paper Organization.* The rest of this article is structured as follows: Section 2 describes the background and our motivation. Section 3 illustrates our proposed approach, Atlas. Section 4 presents our extensive evaluation of Atlas. Section 5 is the discussion. Section 6 is the related works. Section 7 is our conclusion.

## 2 Background and Motivation

This section first presents a brief introduction to compiler optimizations and compiler attributes. Then, we motivate our technique using a real bug example.

### 2.1 Compiler Optimizations

Modern compilers, such as GCC and Clang, use optimization techniques to reduce compilation time, minimize binary size, and improve runtime performance [26]. These compilers typically provide multiple standard optimization levels, such as -O1, -O2, -Os, and -O3, each comprising a set of optimization options. For example, the -O1 optimization level in GCC includes options like `-fdce`, which eliminates unreachable code within the programs, and `-fmerge-constants`, which consolidates duplicate constants within the programs. These optimization options can be applied as part of optimization levels or individually specified in the compilation settings [23]. Whether enabled by default within an optimization level or specified individually, these strategies usually apply to the entire program rather than to specific code elements (e.g., a single function).

Over the past two decades, substantial research efforts have been devoted to the field of compiler testing [6, 18, 29, 31, 34, 37, 38, 54, 66]. Various techniques have been proposed that primarily focus on generating diverse test programs and then utilizing different optimization levels or employing various combinations of optimization options to perform differential testing of compilers. However, these approaches overlook the application of attributes, which allow for the flexible specification of different compilation strategies for individual code elements. As a result, they miss the opportunity to explore the extensive compilation space thoroughly.

### 2.2 Compiler Attributes

Attributes provide a mechanism for passing additional details about code generation and code optimization to the compiler [1]. For instance, attributes can control inlining decisions, specify memory alignment requirements, or impose specific constraints on variable usage [13, 15]. Unlike compiler options, attributes operate at a finer granularity, allowing precise control over specific program elements, such as individual functions or variables. Common types of attributes include function attributes [13], variable attributes [15], and type attributes [14], covering functionalities

```
struct S {
  int i;
};
inline struct S bar() {
  struct S s = { 0 };
  return s;
}
__attribute__((optimize("-ftree-loop-
    distribution")))
void foo() {
  bar();
}
```

Fig. 1. GCC bug 43234. The optimization attribute `optimize(''-ftree-loop-distribution'')` specified for the function `foo()` triggers a GCC bug.

```
int **a;
__attribute__((always_inline)) int *foo
    (void) {
  int b[1];
  (void) b;
}
__attribute__((no_sanitize_address))
    static char
bar(void) {
  *a = foo();
}
void baz(void) {
  bar();
}
```

Fig. 2. GCC bug 114956 which is triggered by a test program generated by Atlas. When this generated program is compiled with the flags `-O2 -fsanitize=address,null`, it results in a crash.

such as optimization control, memory alignment, warning suppression, and specifying function characteristics. For instance, the `noreturn` attribute indicates that a function does not return, while the `pure` attribute specifies that a function has no side effects [13]. Such fine-grained functionality cannot be achieved through compilation options alone.

During compilation, the compiler identifies and parses these attributes in the syntax analysis phase. In the semantic analysis phase, it adjusts its behavior based on the information provided by the attributes. For instance, recognizing a `noreturn` attribute allows the compiler to handle functions that do not return appropriately. In the optimization phase, attributes can guide the compiler to perform specific optimizations or disable certain optimizations. Ultimately, in the code generation phase, the compiler produces the target code following these attributes. By applying different attributes to finer-grained program elements, such as functions or variables, the compiler can trigger additional behaviors, potentially exploring previously unexamined areas within compilers.

As depicted in Figure 1, the highlighted code snippet demonstrates the use of an optimization attribute applied to the `foo()` function. The `-ftree-loop-distribution` optimization option, which can improve cache performance on big loop bodies and allow further loop optimizations [23], is typically enabled at the `-O3` optimization level. When used in conjunction with `attribute`, this optimization can be selectively applied to individual functions. In this case, when GCC compiled the program with the optimization attribute applied to the `foo()` function, a crash occurred. Interestingly, the original program does not have an optimization attribute on `foo()`, and compiling the original program with the optimization option directly did not result in a compiler crash. This approach allows for finer-grained compilation control over individual functions within the program.

Proper use of attributes is a vital strategy for improving both code quality and compiler efficiency. Through the judicious application of attributes, programmers can achieve substantial gains in performance, and clarity of code intent, making attributes an essential tool for enhancing the quality of code and the performance of the compiler.

### 2.3 Motivation

To illustrate the motivation behind our work, consider the compilable program shown in Figure 2. It is generated by our tool, Atlas, which triggered an **Internal Compiler Error (ICE)** when GCC-15 compiled it with `-O2 -fsanitize=address,null`. The `-fsanitize=address` uses AddressSanitizer (ASan) [53] to detect memory-related bugs in programs, and the option `-fsanitize=null` is used to detect null pointer in programs. The highlighted code of the figure denotes the attribute segments inserted by Atlas. While the original program compiled normally, the introduction of these attributes caused the compiler to crash during the compilation of the modified program. The `always_inline` attribute is used to force the compiler to inline a function, even if the compiler's optimization heuristics would normally choose not to do so. The `no_sanitize_address` attribute is used to instruct the compiler not to instrument a particular function with ASan marks, even if ASan is enabled globally.

In this case, when the `always_inline` function `foo()` is inlined into the `no_sanitize_address` function `bar()`, ASan marks (like `.ASAN_MARK` calls) are still left in the code during compiler optimization (`O2`). These leftover marks cause issues during later compilation stages, resulting in an ICE. The compiler does not correctly remove the ASan instrumentation calls when inlining functions into a caller with sanitization disabled, leading to this bug. When attempting to inline the `foo()` function via the command line, the program must be compiled with the `-finline-functions` option. However, this would apply inlining to all functions within the program. Additionally, compiling the program with the `-finline-functions` and `-fsanitize=address` options, without applying the `always_inline` attribute to `foo()`, does not trigger a compiler crash. The crash is only triggered when the `always_inline` and the `no_sanitize_address` attributes are applied separately to the `foo()` and `bar()`, respectively.

The example demonstrates that the combination of optimizations and attributes can result in uncommon scenarios and expose bugs within the compiler. The occurrence of such situations would not have been possible without the use of attributes. This also serves as evidence for the feasibility of utilizing attributes to achieve fine-grained control over the exploration of the compilation space for compiler testing.

## 3 Approach

To systematically validate compiler reliability, we propose a comprehensive approach focused on exploring the compiler's compilation space by generating challenging program variants. The behavior of a compiler is determined by the complex interplay between source code, compiler attributes, and compilation options, which forms an immense and intricate search space. To effectively uncover potential compiler bugs within this space, we design and evaluate two complementary attribute insertion strategies: an *SS* based on random selection, and an *LGS* that leverages LLMs.

The fundamental workflow of our proposed approach is illustrated in Figure 3. Initially, in step ①, the program $\mathcal{P}$ is parsed to identify potential candidates where attributes could be inserted, forming the candidate set $\mathcal{S}$. This step is tailored to the subsequent strategy. For the *SS*, we collect only the syntactic locations where attributes can be inserted. For the *LGS*, however, we extract complete semantic code snippets with their full context, which are required for analysis by the LLM. The different types of information collected in this step form the candidate set $\mathcal{S}$. Subsequently, in step ②, compiler attributes are inserted into program $\mathcal{P}$ based on $\mathcal{S}$, creating a set of mutated program variants $\mathcal{S}_p$. The *SS* randomly inserts attributes at the candidate locations. In contrast, the *LGS* sends the extracted code snippets to the LLM and inserts attributes based on the model's recommendations. Regardless of the strategy used, each variant $\mathcal{P}'$ in $\mathcal{S}_p$ explores a broader range of code paths within the compiler compared to $\mathcal{P}$. As discussed in Section 2.2, some attributes

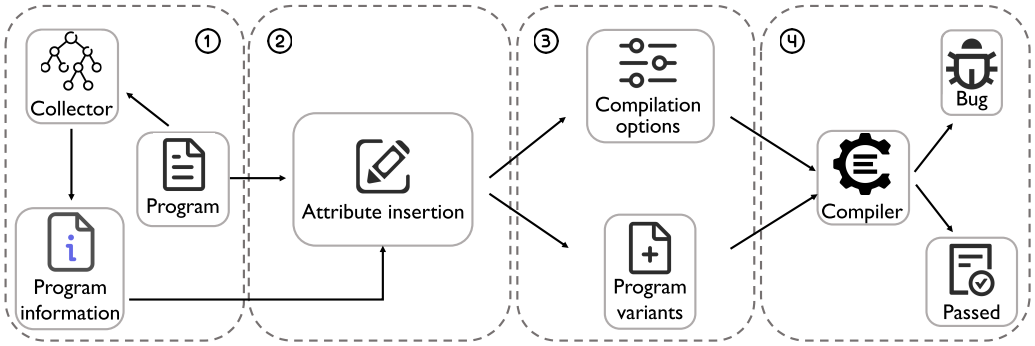



Fig. 3. The workflow of Atlas.

only take effect when specific compilation options are used. To ensure the inserted attributes are effective, we not only insert the attributes but also select the relevant compilation options. In step ③, the strategy for option selection differs according to the attribute insertion method: for variants generated by the *SS*, we utilize a predefined mapping to associate each inserted attribute with its required compilation options, ensuring their activation. For variants produced by the *LGS*, we first validate the compilation options suggested by the LLM for correctness and then apply an iterative de-confliction process to resolve any mutual exclusions among options. This two-step approach guarantees that each attribute is paired with a valid and non-conflicting set of compilation options, enabling the generation of complex and effective compilation scenarios. In step ④, the compilation options selected in step ③ are combined with optimization options to compile the mutated program variants. If the compiler encounters any crashes during the compilation process, the bugs are found.

Our first strategy, the *SS*, operates by randomly inserting compiler attributes at syntactically valid locations, such as before functions, variables, or statements. It acts as a robust baseline capable of effectively discovering a wide variety of compiler bugs. In contrast to the broad exploration of the *SS*, our proposed *LGS* aims for more targeted, in-depth testing. This strategy leverages the powerful code comprehension and reasoning capabilities of LLMs to perform semantic analysis on program code snippets. The LLM does not merely identify syntactically valid insertion points; more importantly, it understands the functionality and context of the code fragment. This allows it to recommend attributes and corresponding compilation options that are semantically relevant and most likely to trigger complex compiler behaviors. This approach is designed to mimic the intuition of a human expert, generating high-quality test cases to reveal deeper compiler bugs.

This section details our approach to combining compilation options and attributes to explore the compilation space for validating compiler reliability. Our strategy attempts to maximize the diversity of compilation option combinations with a wide range of attributes. This approach not only broadens the exploration of the compilation space but also increases the chances of uncovering potential compiler bugs.

### 3.1 Collecting Information

The initial step of our approach involves gathering essential information about functions, variables, and statements present in the test programs. Test programs typically comprise multiple functions and assorted variables. During the front-end parsing phase, the compiler analyzes different code fragments and variables individually and then performs targeted optimizations tailored to their specific requirements. As an illustration, LLVM follows a multistep process for program compilation. Initially, the program undergoes translation into an **Intermediate Representation (IR)**. The

compiler then traverses the IR, identifies specific structural patterns, and applies customized optimizations for those patterns [27, 39]. For instance, the `Instcombine` pass is responsible for identifying duplicate instructions and merging redundant ones through the execution of the pass [44]. Due to the different structure of functions, the compiler can perform different optimizations on them. The granularity and type of information collected depend on the attribute insertion strategy:

—For the *SS*, we identify syntactically valid locations for attribute insertion.
—For the *LGS*, we extract complete semantic code snippets with full context, enabling deeper analysis by the LLM.

*3.1.1 Syntactic Location Collection for Stochastic Insertion.* For our baseline *SS*, the goal is to efficiently identify all syntactically valid points where an attribute can be inserted. Atlas utilizes a lightweight collector to collect function information, variable information, and statement information within the programs. The process is formally defined as follows:

*Definition 1* (Collection). Given a program $\mathcal{P}$ and the function set $\mathcal{S}_f$, variable set $\mathcal{S}_v$, and statement set $\mathcal{S}_s$, the collector $\widehat{Collect}$ records the property tuple $\mathcal{A}$ for each function, variable, and statement in $\mathcal{P}$.

To provide a more detailed explanation, we define the following tuples to depict information from $\widehat{Collect}$:

—$\mathcal{A}_{\langle n,t,pos\rangle}$ contains the following information in $\mathcal{S}_f$: the function name $n$, the function return type $t$, and the function definition locations $pos$.
—$\mathcal{A}_{\langle t,pos\rangle}$ contains the following information in $\mathcal{S}_v$: the variable type $t$, and the variable declaration locations $pos$.
—$\mathcal{A}_{\langle vl,pos\rangle}$ contains the following information in $\mathcal{S}_s$: the variables $vl$ in statement, and the statement locations $pos$.

To acquire insights into the program's structure, Atlas parses the **Abstract Syntax Tree (AST)** of each seed program $\mathcal{P}$ as a unified foundation for both strategies.

*3.1.2 Semantic Snippet Extraction for LLM-Guided Insertion.* Our *LGS* requires a deeper, contextual understanding of a code fragment to be effective. We provide the LLM with the entire, self-contained code snippet for meaningful analysis. To facilitate this, we introduce a more advanced collector, $\widehat{Collect}_{\text{snippet}}$, which identifies the precise start and end boundaries of each targeted code block. By performing a full traversal of the AST, $\widehat{Collect}_{\text{snippet}}$ determines the complete source range of relevant nodes, such as an entire function body, a struct definition, or a loop block including its scope.

*Definition 2* (Semantic Snippet Collection). Given a program $\mathcal{P}$, the collector $\widehat{Collect}_{\text{snippet}}$ records a property tuple for each targeted program element that includes its precise start location, $pos_{\text{start}}$, and code snippet, $snippet$.

The tuples now capture the full boundary of each element:

—$\mathcal{A}_{\langle pos,snippet\rangle}$ contains the following information in $\mathcal{S}_f$: the function definition locations $pos$ and the entire function body $snippet$.
—$\mathcal{A}_{\langle pos,snippet\rangle}$ contains the following information in $\mathcal{S}_v$: the variable start declaration locations $pos$ and the entire variable declaration scope $snippet$.
—$\mathcal{A}_{\langle pos,snippet\rangle}$ contains the following information in $\mathcal{S}_s$: the statement location $pos$ and a complete statement block $snippet$.

```
int a;
int b(int);
<func> int c() {
  a = 1;
  while (a)
    a = 2;
  return a;
}
<func> int e(void) {
  int d = c();
  b(d);
}
```

(a) Original Program $\mathcal{P}_1$.

```
int a;
int b(int);
__attribute__((pure, returns_twice))
int c() {
  a = 1;
  while (a)
    a = 2;
  return a;
}
int e(void) {
  int d = c();
  b(d);
}
```

(b) Mutated Program $\mathcal{P}_f$.

Fig. 4. Illustration examples of function attributes insertion. $\mathcal{P}_f$ was mutated from $\mathcal{P}_1$ by inserted function attribute. This program triggers the GCC bug 114687 when compiled with -fsanitize=address -O1.

For instance, consider the functions in Figure 4(a). While the syntactic collector $\widehat{Collect}$ would record the definition position, the return type (both are *int*), and the variable list (both are *void*) of the function `c()` and `e()`, the new snippet collector $\widehat{Collect}_{\text{snippet}}$ captures the entire function block for `c()` and `e()`. This allows us to pass these code snippets to the LLM for semantic analysis and attribute suggestion.

### 3.2 Inserting Attributes

Having collected potential targets from the source program, the next phase is to generate mutated program variants by inserting compiler attributes. We employ two distinct strategies for this task, each leveraging the specific type of information gathered in the preceding phase. The first strategy introduces attributes at random to explore a diverse range of syntactically valid program variants, while the second utilizes semantic analysis to insert context-aware attributes, aiming to construct more complex test scenarios that are likely to trigger deeper bugs.

*3.2.1 Stochastic Insertion.* As detailed in Section 2.2, attributes allow developers to finely tune their code's behavior and performance, granting them better control over the compilation process. For instance, we can use `__attribute__((optimize(1))) int f();` to instruct the compiler to optimize the function `f()` at the O1 level. In light of this, we propose a method for fine-grained control of compilation by leveraging compiler function attributes. More specifically, we utilize these attributes to explicitly influence the decisions made by the compiler. Algorithm 1 lists the primary procedure used in Atlas to achieve fine-grained control of compilation by randomly inserting attributes into the program.

*Function Attribute Insertion.* For each function within program $\mathcal{P}$, Atlas employs $FlipCoin()$ to probabilistically determine the inclusion of each attribute from the attribute set $\mathcal{S}_{fa}$ (Lines 5–9 in Algorithm 1). The $formFuncAttr()$ method selects one or more attributes and Atlas inserts them into each function (Line 7 in Algorithm 1). Figure 4 illustrates an example of how Atlas inserts function attributes. First, Atlas identifies the location of function definitions based on position information *pos* from $\mathcal{A}_{\langle n,t,pos\rangle}$ marking these locations with <func>. After marking the locations, Atlas selects appropriate function attributes based on the function return type $t$ and inserts them at the specified positions. For instance, the return type of `c()` is `int`, we cannot apply `noreturn` attribute to it. Since $\mathcal{P}_1$ contains two function definitions, two corresponding marked locations are created. The mutated program $\mathcal{P}_f$ shown in Figure 4(b) is derived from $\mathcal{P}_1$, with the highlighted code segments representing the function attributes inserted by Atlas.

**Algorithm 1:** Insert Attributes

**Input:** $\mathcal{P}$, original program
$\mathcal{S}_{fa}$, function attribute set
$\mathcal{S}_{va}$, variable attribute set
$\mathcal{S}_{sa}$, statement attribute set
$\mathcal{S}_f$, function set of the original program
$\mathcal{S}_v$, variable set of the original program
$\mathcal{S}_s$, statement set of the original program
$\mathcal{N}$, the number of mutated programs
**Output:** $\mathcal{S}_p$, the set of programs with attributes inserted

```
1  S_p ← [ ];
2  P' ← P;
3  n ← 0;
4  while true do
5      foreach A_⟨n,t,pos⟩ ∈ S_f do
6          if FlipCoin() then
7              attr ← formFuncAttr(S_fa, A_⟨n,t,pos⟩);
8              pos ← A_⟨n,t,pos⟩.getPos();
9              P' ← insertAttr(P', attr, pos);
10     foreach A_⟨t,pos⟩ ∈ S_v do
11         if FlipCoin() then
12             attr ← selectVarAttr(S_va, A_⟨t,pos⟩);
13             pos ← A_⟨t,pos⟩.getPos();
14             P' ← insertAttr(P', attr, pos);
15     foreach A_⟨vl,pos⟩ ∈ S_s do
16         if FlipCoin() then
17             attr ← formStmtAttr(S_sa, A_⟨vl,pos⟩);
18             pos ← A_⟨vl,pos⟩.getPos();
19             P' ← insertAttr(P', attr, pos);
20     S_p.add(P');
21     n++;
22     if n >= N then
23         break
24 return S_p
```

*Variable Attribute Insertion.* For each variable within program $\mathcal{P}$, the method $selectVarAttr()$ picks appropriate variable attributes through type checking (Line 12 in Algorithm 1). For example, the `packed` attribute, which instructs the compiler to pack structure members tightly together without inserting padding [15], cannot be applied to variables of type `int`. As such, the method $selectVarAttr()$ will only apply `packed` attribute to member variables within a structure. In the case of program $\mathcal{P}_2$ shown in Figure 5(a), Atlas identifies the positions of all variables and marks them with `<var>` using the position information $pos$ from $\mathcal{A}_{\langle t,pos\rangle}$. The mutated program $\mathcal{P}_v$ in Figure 5(b) is derived from $\mathcal{P}_2$, where Atlas inserts the `vector_size` attribute into a member

```
<var> int a;
void b();
int e(int *c, struct d {
<var> char an[*c]} *) {
  sizeof(struct d);
}
void f() {
  if (e(&a, 0))
    b();
}
```

(a) Original Program $\mathcal{P}_2$.

```
int a;
void b();
int e(int *c, struct d {
__attribute__((vector_size(4)))
  char an[*c]} *) {
  sizeof(struct d);
}
void f() {
  if (e(&a, 0))
    b();
}
```

(b) Mutated Program $\mathcal{P}_v$.

Fig. 5. Illustration examples of variable attributes insertion. $\mathcal{P}_v$ was mutated from $\mathcal{P}_2$ by inserted variable attribute. $\mathcal{P}_v$ triggers the GCC bug 117145 when compiled with -O1.

```
void a (int &x, int &y) {
  <stmt>
  for (i=0; i<10; i++) {
    x += y + 1;
  }
}
```

(a) Original Program $\mathcal{P}_3$.

```
void a (int &x, int &y) {
  #pragma omp simd linear(uval (x): y+1)
  for (i=0; i<10; i++) {
    x += y + 1;
  }
}
```

(b) Mutated Program $\mathcal{P}_s$.

Fig. 6. Illustration examples of statement attributes insertion. $\mathcal{P}_s$ was mutated from $\mathcal{P}_3$ by inserted statement attribute. This program triggers the LLVM bug 108166.

variable of the structure d. Since there are two `<var>` marks in $\mathcal{P}_2$, Atlas selects the one within the structure d based on the outcome of the *FlipCoin*() decision.

*Statement Attribute Insertion.* The process of statement attribute insertion is outlined in Algorithm 2. This process begins with the use of the function *FlipCoin*() to determine which attributes from the set $\mathcal{S}_{sa}$ will be utilized in subsequent steps (Line 4). The method *isEmpty*() is employed to verify the presence of variables within the statement (Line 6). If variables are indeed present, the *Construct*() method generates a simple expression based on the identified variable list *vl* to accomplish the corresponding attribute. Notably, each statement is associated with only a single attribute. To provide a clearer understanding, consider program $\mathcal{P}_3$ as depicted in Figure 6(a), where Atlas annotates the loop statement with `<stmt>`. The variables x and y are included in the variable list; consequently, the *Construct*() method selects a `linear` modifier and produces simple expression `(uval (x): y+1)`. The highlighted code in Figure 6(b) illustrates the attribute generated by $formStmtAttr()$. `#pragma` is also a form of attribute. For clarity, this line of code has been folded to enhance readability.

*3.2.2 LLM-Guided Semantic-Aware Insertion.* In contrast to the *SS*, our LLM-guided approach leverages semantic context for testing. This strategy utilizes the complete code snippets extracted by the $\widehat{Collect}_{\text{snippet}}$. The core idea is to treat the LLM as a domain expert that can analyze a code fragment and suggest attributes that are not only syntactically correct but also semantically potent for uncovering compiler bugs. The workflow is as follows:

*(1) Prompt Engineering.* The *LGS* depends on providing the model with clear, explicit instructions. To achieve this, we have developed a structured prompt template, shown in Figure 7, which guides the LLM's reasoning process. For each code snippet extracted from the program, we dynamically populate this template to construct a precise query.

**Algorithm 2:** Form Statement Attributes

```
Function formStmtAttr(S_sa, A_⟨vl,pos⟩):
    a ← " ";
    foreach attr ∈ S_sa do
        if FlipCoin() then
            var_list ← A_⟨vl,pos⟩.getVarList();
            if !var_list.isEmpty() then
                a ← Construct(var_list);
                break
            else
                a ← attr;
    return a
```

**Prompt Template for Attribute Generation**

I will provide you with a C/C++ <CODE_TYPE> snippet. Your task is to act as a compiler fuzzing expert.
1. Analyze the given code snippet. Based on your analysis, generate suitable attributes for the <COMPILER_TYPE> compiler. Your goal is to create combinations of attributes that are logically valid and can stress-test the compiler.
2. Ensure every attribute you provide is syntactically correct and semantically applicable to the given code snippet for the target compiler.
3. Provide <NUMBER> diverse attribute combinations. For each combination, also provide the necessary compiler options required to activate it. Use the following format for each combination:

```
attributes: __attribute__((...))
options: <required_compiler_options>
```

Fig. 7. The general-purpose prompt template used to query the LLM for generating semantically relevant attributes and their corresponding compilation options. Placeholders like <CODE_TYPE> are dynamically replaced based on the context.

The template is composed of several key components:

— *Role Assignment:* The prompt begins by assigning the LLM the role of a "compiler fuzzing expert." This instruction primes the model to access its knowledge related to compiler internals, optimizations, and common sources of bugs.
— *Contextual Placeholders:* The template includes several placeholders, such as <CODE_TYPE> (e.g., "function," "loop"), <COMPILER_TYPE> (e.g., "GCC," "Clang"), and <NUMBER> (the desired number of suggestions). These placeholders are then dynamically populated with the specific details of the code snippet under analysis, tailoring the query for each unique context.
— *Core Task Directives:* The prompt contains explicit directives. It instructs the model to analyze the code (Directive 1), ensure the correctness and applicability of its suggestions (Directive 2), and provide a diverse set of attribute-option pairs (Directive 3). This multi-step instruction encourages a more thorough and reliable analysis than a simple, open-ended question.

**Algorithm 3:** LLM-Guided Attribute Insertion

**Input:** $\mathcal{P}$, original program
$\mathcal{S}_{snip}$, set of code snippets from $\mathcal{P}$
$LLM$, a Large Language Model API
**Output:** $\mathcal{S}_p$, the set of programs with LLM-suggested attributes

```
1  S_p ← [ ];
2  foreach snippet ∈ S_snip do
3      prompt ← formPrompt(snippet);
4      response ← queryLLM(LLM, prompt);
5      parsed_data ← parseResponse(response);
6      if parsed_data is valid then
7          attr ← parsed_data.getAttribute();
8          pos ← snippet.getPos();
9          P' ← insertAttr(P, attr, pos);
10         P'.setRecommendedOptions(parsed_data.getOptions());
11         S_p.add(P');
12 return S_p
```

— *Strict Formatting Constraint:* Finally, the prompt requires a specific "attributes: ... options: ..." format for the output. This constraint is crucial for ensuring that the LLM's response can be reliably and automatically parsed by our tool, eliminating the need for complex natural language processing on the output.

By using this carefully engineered template, we transform the task of attribute generation from a random process into a guided, deterministic query, significantly improving the semantic relevance of the generated test cases.

*(2) LLM Inference and Response Parsing.* The constructed prompt is sent to a pre-trained LLM. We then parse the model's structured response to extract the recommended attribute string and the list of associated compilation options.

*(3) Attribute Insertion.* Once the attribute is extracted, it is inserted into the original program $\mathcal{P}$ at the start of the code snippet's location. The resulting mutated program $\mathcal{P}'$ now contains a semantically informed attribute.

This entire process is formalized in Algorithm 3. The input of the algorithm includes the original program $\mathcal{P}$, a set of code snippets $\mathcal{S}_{snip}$ extracted during the information collection phase, and an interface to an LLM API. Its output is a set of mutated program variants $\mathcal{S}_p$.

The algorithm iterates through each `snippet` in the set $\mathcal{S}_{snip}$ (Line 2). For every snippet, it first constructs a tailored query by calling `formPrompt()` (Line 3), which populates our prompt template (Figure 7) with the specific details of the code snippet. This prompt is then sent to the LLM via the `queryLLM()` function, and the model's raw response is retrieved (Line 4).

The `parseResponse()` function (Line 5) processes the LLM's output, expecting it to conform to the structured format we requested. If the response is valid and successfully parsed, the algorithm proceeds to extract the suggested attribute and its corresponding options (Lines 6–11). The `getAttribute()` method retrieves the attribute string, while `getPos()` obtains the starting location of the snippet. The core mutation happens in Line 9, where `insertAttr()` inserts the attribute into a copy of the original program, creating a new variant $\mathcal{P}'$.

**Algorithm 4:** Select Compilation Options for the SS

**Input:** $\mathcal{P}$, the programs with attributes inserted
$\mathcal{M}_o$, the map of attributes and their options
$\mathcal{M}_{rules}$, a predefined set of mutual exclusion rules for options in $\mathcal{M}_o$
**Output:** $O$, the compilation option of $\mathcal{P}$

```
1 select ← set();
2 L_attr ← getInsertedAttr(P);
3 foreach attr ∈ L_attr do
4     if attr ∈ M_o.keys() then
5         select.add(M_o.getValues(attr));
6 O ← formOpt(select, M_rules);
7 return O
```

Significantly, the compiler options recommended by the LLM are not discarded. They are associated with the newly created program variant $\mathcal{P}'$ using the `setRecommendedOptions()` method (Line 10). This ensures that the semantic link between the attribute and its required flags is preserved for the subsequent compilation and validation phase. Finally, the successfully mutated program $\mathcal{P}'$ is added to the output set $\mathcal{S}_p$ (Line 11). This process is repeated for all snippets, ultimately producing a collection of program variants, each enhanced with a semantically targeted attribute suggested by the LLM.

### 3.3 Selecting Options

In practical use, certain attributes become effective only when specific compilation options are applied. As illustrated in Figure 2, we utilize the `no_sanitize_address` attribute to disable the instrumentation of `bar()` function. Such attribute will only exert its influence during the compilation process if the `-fsanitize=address` option is specified. Therefore, following the insertion of attributes, selecting the appropriate compilation options is a crucial step to ensure their effectiveness and create complex compilation scenarios. The procedure for this selection differs significantly depending on whether the variant was produced by our stochastic or LLM-guided strategy.

*3.3.1 Option Selection for the SS.* For program variants generated via the *SS*, the selection process relies on a predefined, static mapping to ensure attribute effectiveness. The procedure is outlined in Algorithm 4. In the map $\mathcal{M}_o$, each attribute (key) corresponds to the compilation options (value) that activate it. The $getInsertedAttr()$ method retrieves all attributes that have been inserted into the program $\mathcal{P}$ (Line 2). By iterating through the attributes in the list $\mathcal{L}_{attr}$, the corresponding compilation options are retrieved (Lines 3–5). The $formOpt()$ function (Line 6) then consolidates these options and performs a crucial de-confliction step. This step is straightforward because it operates on a known and controlled set of options. All potential conflicts (e.g., `-fsanitize=address` and `-fsanitize=thread` being mutually exclusive) are known in advance and are encoded in a predefined set of mutual exclusion rules, $\mathcal{M}_{rules}$. The de-confliction process simply applies these static rules to resolve any issues, ensuring a valid final option set.

*3.3.2 Option Selection for the LGS.* In contrast, variants from the LGS require a more sophisticated approach. The LLM may generate novel combinations of known options or suggest less common, "unknown" options that are not in our pre-defined map $\mathcal{M}_o$. The static de-confliction

**Algorithm 5:** Dynamic De-Confliction of LLM-Suggested Options

**Input:** $O_{valid}$, the validated set of options from the LLM
$M_{rules}$, a set of mutually exclusive option pairs
$\mathcal{P}$, original program
**Output:** $O$, the de-conflicted set of options

```
1  O ← O_valid;
2  while true do
3      C_pairs ← ∅;
4      foreach (opt_A, opt_B) ∈ M_rules do
5          if opt_A ∈ O and opt_B ∈ O then
6              C_pairs.add((opt_A, opt_B));
7      if C_pairs is not empty then
8          (opt_A, opt_B) ← selectRandomPair(C_pairs);
9          opt_remove ← randomChoice(opt_A, opt_B);
10         O ← O \ {opt_remove};
11         continue;
12     result ← attemptCompile(P, O);
13     if result is success then
14         break;
15     else
16         (opt_X, opt_Y) ← identifyConflict(O);
17         M_rules.add((opt_X, opt_Y));
18         opt_remove ← randomChoice(opt_X, opt_Y);
19         O ← O \ {opt_remove};
20 return O
```

method of Algorithm 4 is inherently unsuitable for this dynamic and unpredictable scenario. Therefore, we do not trust the LLM's suggestions unconditionally; instead, we subject them to a rigorous two-step validation and de-confliction process.

*Step 1: Validity Checking.* The initial set of options suggested by the LLM for a given attribute is first validated for basic correctness. We attempt to compile the original, unmodified program $\mathcal{P}$ using each option individually. If the compilation fails, it indicates that the option itself is invalid, and this option is discarded. Only options that pass this basic check proceed to the next step.

*Step 2: Iterative De-Confliction.* A validated set of options may still contain internal conflicts. To resolve these, we employ a systematic, rule-based iterative de-confliction algorithm, detailed in Algorithm 5. It is designed to find a maximal non-conflicting subset of the LLM's suggestions.

The de-confliction process begins with the set of validated options, $O_{valid}$, and our dynamically updated set of mutual exclusion rules, $M_{rules}$. The algorithm then enters a loop that continues until no conflicts remain (Line 2).

In each iteration, the algorithm first identifies all known conflicting pairs that are currently present in the working set $O$ by checking against the rules in $M_{rules}$ (Lines 4–6). If any known conflicts are found ($C_{pairs}$ is not empty), the algorithm resolves one such conflict by randomly selecting a pair and removing one of its options (Lines 8–11) and then continues to the next iteration. Only when all known conflicts are resolved, does the algorithm proceed to check for unknown conflicts. It attempts to compile the program with the entire set of remaining options (Line 12). If

compilation succeeds, this indicates no conflicts remain, and the algorithm terminates (Line 14). However, if the compilation fails, this indicates an unknown conflict exists within the set. The algorithm then identifies the conflicting pair by parsing compiler error messages or using a binary search approach on the option set (Line 16). It then adds this newly discovered conflict to $M_{rules}$ (Line 17) and removes one of the conflicting options (Lines 18–19). This dynamic rule-updating mechanism allows the system to adapt to the novel option combinations provided by the LLM. This dynamic, adaptive approach allows us to handle the unpredictable but potentially valuable option sets generated by the LLM. The result of this process is a final, non-conflicting base set of options, $O$, ready for the next phase.

### 3.4 Validating Compiler

Once the program variants are generated, the testing process becomes straightforward. The modifications made by Atlas, such as attribute insertion, do not compromise the overall structure of the programs. For each program variant $\mathcal{P}'$, we begin with the base set of compilation options, $O$, which was derived in the previous phase. The origin of this set depends on the strategy used to generate the variant:

- For a variant from the *SS*, $O$ consists of the options retrieved from our predefined map, corresponding to the randomly inserted attributes.
- For a variant from the *LGS*, $O$ is the final, de-conflicted set of options that were suggested by the LLM and subsequently validated by our de-confliction algorithm.

For each variant, a stochastic selection process is employed to randomly choose a sequence of optimization options from a set of options $O_{all}$. $O_{all}$ contains all optimization options for the compiler. The number of selected optimization options does not exceed five, as previous research investigations have indicated that the average number of optimization options leading to bugs is five [6]. These selected options are then combined with the options in $O$ determined by Algorithm 4 for compilation along with the program variant. We define the validation process as follows:

*Definition 3* (Validation). Let program $\mathcal{P}_i$ be a variant of program $\mathcal{P}$ and the option $O_i$ be the corresponding carefully selected options. The compiler under test is denoted as $Comp$, which can receive compilation options and test programs as inputs. If $Comp(\mathcal{P}_i, O_i)$ crashed, a compiler bug is identified.

Our key insight is that when considering a valid program $\mathcal{P}$, the compiler should exhibit stability without encountering any crashes throughout the compilation process with a specific compilation option, denoted as $O_p$. Figure 2 illustrates the final results of our tool. In the case of the attribute-inserted variant, detecting the bug requires both the selection of appropriate compiler options (`-fsanitize=address,null`) and the application of suitable optimization settings (`-O2`). The results illustrated in Figures 4 and 5 further substantiate the importance of selecting the appropriate optimization options in the identification of compiler bugs.

## 4 Evaluation

This section presents our extensive evaluation of Atlas to find bugs in compilers. Our evaluation is based on the following research questions:

- *RQ1 (Effectiveness):* How many compiler bugs have been found by Atlas?
- *RQ2 (Efficiency):* How efficient is Atlas in uncovering compiler bugs?
- *RQ3 (Capability):* Can Atlas improve code coverage?

### 4.1 Implementation and Evaluation Setup

*Implementation.* Atlas is designed to run in a fully automated way, including program information collection, attribute-guided mutation, and validation stages. Atlas utilizes LLVM's LibTooling library [41] to facilitate the program information collection task as described in Section 3.1. For our LGS, we leverage the deepseek-v3 LLM through its API [12]. This model excels at tasks like code generation and analysis. To promote factual and repeatable suggestions for coding tasks, the API is queried with a low temperature setting of 0, which is recommended in the documentation [11]. Once launched, Atlas will automatically generate programs and feed them into the compilers under test.

*Target Compilers.* We used Atlas to continuously test the trunk version of two widely used compilers: GCC and LLVM. Specifically, we obtained daily builds of GCC from its repositories and utilized nightly packages of LLVM [42]. Testing on the latest development versions of software offers several advantages:

— By testing these trunk versions, developers can immediately identify and address these bugs.
— This approach can avoid duplicate bug reports as much as possible because once bugs are found in the trunk version, developers tend to quickly fix them [29, 65].

*Test Programs.* In our testing, we have drawn from two sources of programs to generate test cases:

— *Compiler Test Suites*: Each of GCC and LLVM has an already sizable and expanding regression test suite [22, 40], which we can use for generating various test programs. We chose GCC test suite and LLVM test suite because: (1) they are open source which can be easily obtained; (2) they are typically used for compiler regression testing, covering a wide range of C semantics; and (3) many of these test programs can be compiled and executed independently, as they do not rely on external libraries and have fixed inputs.
— *Generated Code*: We chose Csmith [65] and YARPGen [37, 38] as the random program generators to generate C programs to test compilers for the following main reasons: (1) they are extensively used in the literature of C compiler testing [6, 30, 54]; (2) they are effective in finding bugs as thousands of bugs have been exposed and reported for the most widely used C compilers; (3) each test case generated by Csmith and YARPGen is valid and does not require external inputs; and (4) they are efficient as a test program with tens of thousands of lines can be generated quickly.

*Attributes.* For the *SS*, all the attributes we used are from the GCC and LLVM documentation [16, 20]. The attributes and options map $\mathcal{M}_o$ described in Algorithm 4 was created based on the official documentation. Likewise, the mutual exclusion rules $M_{rules}$ described in Algorithm 5 are also sourced from this documentation. We chose attributes for the following main standard: (1) Programming Language Compatibility: The attribute must be compatible with the C programming language. (2) Exclusions: Attributes that are specifically designed for GPU support are not considered. The attribute sets utilized for GCC and LLVM in our experiments were distinct.

*Hardware.* Our evaluation was conducted on a workstation with an Intel 48-core 2.30 GHz CPU, 120GiB RAM, and Ubuntu 22.04.2 LTS operating system.

*Testing Process.* Our testing process is fully automated and runs continuously. Atlas first inserts attributes into programs for the test program to get the program variants set and then chooses a proper compilation option for each variant. The program variant with a range of compilation options is used to test GCC and LLVM. Once a program triggers a compiler bug, we use C-Reduce [52] to reduce the program. If the program triggers a new compiler bug, we report the bug to the developers. The only manual step in the testing is analyzing and reporting bugs.

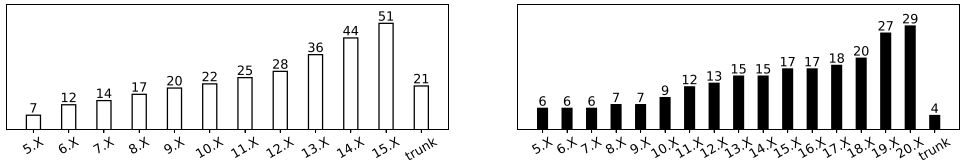
Table 1. The Bugs Reported by Atlas

| ID | Pri. | Status | Opt. | Attributes | ID | Pri. | Status | Opt. | Attributes |
|---|---|---|---|---|---|---|---|---|---|
| The reported bugs of GCC | | | | | | | | | |
| 112709 | **P2** | Fixed | ✓ | returns_twice | 117358 | **P2** | Fixed | ✓ | const(func) |
| 112860 | P3 | Fixed | ✓ | vector_size(var) | 117380 | P4 | Confirmed | ✗ | vector_size(var) |
| 113043 | P3 | Confirmed | ✓ | interrupt | 117429 | P3 | Duplicate | ✓ | no_sanitize,flatten |
| 113264 | P3 | Confirmed | ✗ | alias,sanitize,target_clones,copy | 117437 | P3 | Confirmed | ✓ | sanitize |
| 114687 | **P1** | Fixed | ✓ | pure, returns_twice | 117440 | **P2** | Confirmed | ✓ | pure |
| 114956 | **P2** | Fixed | ✓ | always_inline,sanitize | 117461 | P3 | Unconfirmed | ✓ | returns_twice |
| 115509 | P3 | Unconfirmed | ✗ | aligned(func),sentinel | 117479 | P5 | Confirmed | ✗ | vector_size(func) |
| 115530 | P3 | Confirmed | ✓ | simd(func) | 117489 | **P2** | Confirmed | ✓ | pure,sanitize |
| 115548 | P3 | Unconfirmed | ✗ | malloc,simd(func) | 117505 | P3 | Confirmed | ✗ | used |
| 115549 | P3 | Fixed | ✗ | aligned(func),optimize(s) | 117512 | **P2** | Fixed | ✗ | aligned(var) |
| 115573 | P3 | Unconfirmed | ✓ | no_reorder | 117540 | P3 | Duplicate | ✓ | flatten |
| 115595 | P3 | Unconfirmed | ✓ | optimize(1),noipa | 117979 | **P2** | Fixed | ✓ | returns_twice,const(func) |
| 115815 | **P2** | Fixed | ✓ | destructor | 120839 | **P1** | Confirmed | ✓ | aligned |
| 115816 | P3 | Confirmed | ✓ | vector_size(var),target | 120844 | P3 | Unconfirmed | ✓ | simd(stmt) |
| 115847 | P3 | Confirmed | ✗ | simd(func),vector_size(func) | 120862 | P3 | Confirmed | ✓ | aligned |
| 115848 | P3 | Assigned | ✓ | strub | 120911 | P3 | Unconfirmed | ✗ | weakref |
| 115859 | P3 | Confirmed | ✓ | optimize(3) | 120913 | P3 | Confirmed | ✓ | vector_size |
| 115861 | P3 | Unconfirmed | ✗ | deprecated | 120971 | P3 | Unconfirmed | ✓ | simd(stmt) |
| 116607 | P3 | Fixed | ✓ | sanitize | 120973 | P3 | Confirmed | ✓ | naked |
| 116612 | P3 | Duplicate | ✗ | aligned(var) | 120991 | P3 | Unconfirmed | ✗ | transaction_safe |
| 116659 | P3 | Duplicate | ✓ | optimize("Ofast") | 120992 | P3 | Confirmed | ✓ | may_alias |
| 116687 | P3 | Unconfirmed | ✓ | simd(func) | 121032 | P3 | Confirmed | ✓ | simd, vector_size |
| 117142 | **P2** | Fixed | ✓ | returns_twice | 121081 | P3 | Confirmed | ✗ | visibility |
| 117145 | **P2** | Fixed | ✓ | vector_size(var) | 121111 | P3 | Confirmed | ✗ | vector_size |
| 117167 | P3 | Duplicate | ✓ | const(func) | 121119 | P3 | Unconfirmed | ✓ | simd(stmt) |
| 117197 | P3 | Confirmed | ✗ | vector_size(var) | 121137 | P3 | Confirmed | ✓ | simd(stmt) |
| 117209 | **P2** | Fixed | ✓ | returns_twice,pure | 121181 | P3 | Confirmed | ✗ | destructor |
| 117230 | **P2** | Fixed | ✓ | vector_size(var) | 121187 | P3 | Unconfirmed | ✓ | simd(stmt) |
| 117245 | **P2** | Assigned | ✗ | vector_size(var) | 121188 | P3 | Unconfirmed | ✓ | simd(stmt) |
| 117254 | **P2** | Fixed | ✗ | nonstring | 121189 | P3 | Unconfirmed | ✗ | vector_size, target_clones |
| 117295 | P3 | Unconfirmed | ✓ | returns_twice | 121193 | P3 | Duplicate | ✗ | may_alias |
| 117326 | P3 | Confirmed | ✓ | constructor | 121211 | P3 | Confirmed | ✓ | simd(stmt) |
| The reported bugs of LLVM | | | | | | | | | |
| 73378 | - | Confirmed | ✓ | noinline | 113401 | - | Confirmed | ✓ | preserve_most |
| 73797 | - | Duplicate | ✓ | noinline | 113403 | - | Confirmed | ✓ | speculative_load_hardening |
| 74189 | - | Fixed | ✓ | packed | 113420 | - | Confirmed | ✓ | target |
| 75019 | - | Confirmed | ✗ | noinline | 115299 | - | Fixed | ✗ | cpu_specific |
| 75301 | - | Confirmed | ✓ | vector_size | 115411 | - | Fixed | ✓ | optnone |
| 75321 | - | Confirmed | ✓ | loop_vectorize(stmt) | 115490 | - | Confirmed | ✗ | no_caller_saved_registers |
| 75690 | - | Duplicate | ✓ | vector_size | 115602 | - | Confirmed | ✓ | cold |
| 88692 | - | Confirmed | ✓ | minsize,sanitize | 115632 | - | Confirmed | ✓ | address_space |
| 95892 | - | Confirmed | ✗ | opencl_global_device | 115655 | - | Confirmed | ✗ | vector_size |
| 95928 | - | Confirmed | ✓ | preserve_none | 116010 | - | Confirmed | ✓ | alias,constructor |
| 96018 | - | Confirmed | ✗ | interrupt | 116124 | - | Fixed | ✗ | noderef |
| 96383 | - | Confirmed | ✓ | simd(stmt) | 121865 | - | Confirmed | ✗ | vectorcall |
| 98635 | - | Confirmed | ✗ | regcall | 131488 | - | Confirmed | ✓ | ifunc |
| 98663 | - | Confirmed | ✗ | vectorcall | 150362 | - | Confirmed | ✗ | address_space, aligned |
| 107979 | - | Confirmed | ✓ | atomic | 150363 | - | Confirmed | ✓ | address_space |
| 108166 | - | Confirmed | ✓ | simd(stmt) | 150384 | - | Confirmed | ✗ | vector_size |
| 113285 | - | Confirmed | ✓ | nodebug | - | - | - | - | - |

## 4.2 RQ1: Bug-Finding

Table 1 summarizes the bugs we discovered during our testing period. Overall, we reported 97 bugs in total: 64 for GCC and 33 for LLVM. The developers have confirmed 73 of them. This highlights the bug-finding capability of Atlas. Of all these bugs, 19 of them have been fixed of which 15 fixed bugs are in GCC and 4 fixed bugs are in LLVM.[2] We also observed that the LLVM developers were less responsive than those of GCC and mostly only labelled our reports as bugs without further diagnosis. The results presented in the "Opt." column indicate whether the triggering of a

[2]For one GCC bug (ID: 116687), developers commented it as "Confirmed," unless they marked them as "NEW," we still regarded it as unconfirmed.




(a) Affected GCC versions (b) Affected LLVM versions

Fig. 8. Compiler versions affected by the reported bugs.

bug requires compilation options. If such options are necessary, the bug is marked with ✓; if not, it is marked with ✗. A total of 49 bugs require compilation options for triggering, while only 24 bugs can be triggered solely through the insertion of attributes. In Table 1, the "Pri." is priority of a bug report. The field priority of a bug report is determined by developers for prioritizing bugs, expressing the order in which bugs should be fixed [55]. P1 is the most urgent level and bugs of this level should be fixed soon, and P3 is the default priority level. LLVM developers do not prioritize bugs explicitly in the bug repository, so we only present the priority on GCC.

Note that the bug management guidelines of GCC and LLVM are not exactly similar. Specifically, a reported bug in GCC Bugzilla will be initially labelled as "UNCONFIRMED," and the status will turn to "NEW" if developers confirm it. However, in LLVM issue tracker, a reported bug will be labelled as "new issue" by default, and if the developers confirm it, they will remove the "new issue" label and add other labels. For example, if the bug is related to link time optimization, it will be labelled as "LTO." In addition, if the status of a bug is "Assigned" means the bug is confirmed and is assigned to a certain developer to deal with it. If a bug is fixed by developers in LLVM issue tracker, the corresponding issue report will be closed. If the bug has the same root cause with the bug reported by other developers previously, we categorize it as "Duplicate."

*Priority of Bugs.* To analyze the importance of the bugs, we present the priority levels of GCC bugs in Table 1. Two bugs (ID:114687 and 120839) have been marked as P1 priority by the developers, and 14 bugs have been marked as P2 priority. The higher priority assigned to these bugs indicates the developers' attention to the bugs. Developers have to fix all P1 bugs before making the next release [34].

*Influence of Bugs.* To further understand the influence of our reported bugs, we investigate the official release versions of compilers affected by the reported bugs. Here, we select GCC-5 (released in 2015) and LLVM-5 (released in 2017) as the earliest versions of the subjects because they are the first stable versions that support sanitizers [33] and our compilation options include sanitize options. The results indicate that Atlas is capable of identifying critical bugs. As illustrated in Figure 8, four enduring bugs spanning 8 years are evident within the GCC-5, signifying its persistence over a considerable period. Similarly, in LLVM, four bugs have persisted within the compiler for a duration of 6 years. Most bugs affect at least two compiler versions. The significance of these findings is further underscored by direct feedback from compiler developers. For instance, in the bug report for GCC Bug 121211, a developer validated the long-lived nature of the bug by confirming its origin:

*—Looks like this has been an issue since openmp simd aligned support was added in GCC 4.9.0.*

This comment is particularly revealing, as it indicates that the bug is not a subsequent regression but an error in the original implementation of the `openmp simd aligned` feature in GCC 4.9.0. In other words, this functionality was flawed from its very inception, a critical issue that remained hidden for a decade until uncovered by Atlas. The ability to uncover such deep-seated bug demonstrates the practical impact and effectiveness of our approach in enhancing compiler reliability.

Table 2. The Number of Affected Compiler Components

| Component | GCC | LLVM | Total |
|---|---|---|---|
| Front-End | 31 | 6 | 37 |
| Middle-End | 16 | 8 | 24 |
| Optimization | 12 | 7 | 19 |
| Back-End | 5 | 12 | 17 |

*Severity of the Bugs.* Some of the identified bugs have an impact on compiler correctness and reliability. For example, GCC Bug 112709 leads to an ICE during sanitizer instrumentation on function return values, which directly causes the compiler to crash. GCC Bug 114687 reflects an overly aggressive constraint in handling the function call during instrumentation, which may result in incorrect control-flow or optimization behavior. These issues demonstrate fragility in the compiler's Static Single Assignment and Control Flow Graph transformation logic, especially under instrumentation. Other bugs affect semantic correctness or type handling. For instance, GCC Bug 117145 occurs when a vector_size attribute is applied to a struct, and the compiler fails to treat it as a Variable Modified Type, leading to type analysis errors. GCC Bug 117230 misuses type precision on vector types, potentially resulting in incorrect code generation or undefined behavior when vector types are involved. Bugs related to SIMD support can result in broken vectorization or compilation failure. GCC Bug 115847 misinterprets the simd attribute on functions with vector return types, breaking code generation. LLVM Bugs 96383 and 108166 show the compiler's inability to handle loop-level SIMD attributes properly, which causes compilation errors when auto-vectorization is attempted. These examples represent only a subset of the bugs we identified. Many other issues were found across different components of the compiler. Overall, these bugs impact the reliability of compilers in real-world scenarios.

*Affected Compiler Components.* As shown in Table 2, the 97 identified bugs span a wide range of compiler components, with the front-end being the most affected, accounting for 37 bugs (39.4%). Furthermore, a significant number of bugs were found in the middle-end (24 bugs) and dedicated optimization components (19 bugs). This demonstrates that attribute insertion effectively stresses the compiler's core transformation and optimization logic, influencing the generation and manipulation of IRs. This broad component coverage validates the effectiveness of our strategies in generating diverse and challenging test cases. Crucially, finding bugs in both the initial parsing stages (front-end) and the final machine code generation phase (back-end) confirms that the impact of attribute manipulation persists throughout the full compilation process. This ability to stress all major compiler stages underscores the value of our attribute-guided approach in thoroughly exploring the compiler's complex behavior.

*Valuable Attributes.* The attribute combination of each bug is listed in Table 1. Since some attributes can be applied to both functions and variables, we explicitly annotate their usage: "func" represents function attribute, "var" represents variable attribute, "stmt" represents statement attribute. To ascertain the effectiveness of various attributes in bug detection, we analyzed the 97 test cases that trigger bugs. Specifically, we find that each test case contained at least one attribute, indicating that attribute insertion is essential. Delving deeper, we find that certain attributes are more frequently involved in bug-triggering test cases. Table 3 presents the number and types of attributes involved in the test cases. The attributes of GCC and LLVM displayed in the table reveal differences. For example, in the context of optimization-related attributes, LLVM and GCC have distinct approaches to applying optimization settings to functions. LLVM typically disables optimization for a function

Table 3. The Distribution of the Valuable Attributes

| Compiler | Attribute | Type | # | Attribute | Type | # |
|---|---|---|---|---|---|---|
| GCC | vector_size | var. | 10 | copy | func. | 2 |
| | simd | stmt. | 9 | destructor | func. | 2 |
| | return_twice | func. | 7 | flatten | func. | 2 |
| | sanitize | func. | 5 | malloc | func. | 2 |
| | aligned | var. | 4 | may_alias | var. | 2 |
| | optimize | func. | 4 | target | func. | 2 |
| | pure | func. | 4 | alias | func. | 1 |
| | simd | func. | 4 | aligned | stmt. | 1 |
| | const | func. | 3 | always_inline | func. | 1 |
| | vector_size | func. | 3 | constructor | func. | 1 |
| | aligned | func. | 2 | deprecated | func. | 1 |
| | interrupt | func. | 1 | naked | func. | 1 |
| | nonstring | var. | 1 | noipa | func. | 1 |
| | no_reorder | func. | 1 | sentinel | func. | 1 |
| | strub | func. | 1 | target_clones | func. | 1 |
| | transaction_safe | func. | 1 | used | func. | 1 |
| | visibility | var. | 1 | weakref | func. | 1 |
| LLVM | vector_size | var. | 4 | interrupt | func. | 1 |
| | address_space | var. | 3 | loop_vectorize | stmt. | 1 |
| | noinline | func. | 3 | minsize | func. | 1 |
| | simd | stmt. | 2 | nodebug | func. | 1 |
| | vectorcall | func. | 2 | noderef | var. | 1 |
| | alias | func. | 1 | opencl | func. | 1 |
| | aligned | var. | 1 | optnone | func. | 1 |
| | atomic | func. | 1 | packed | var. | 1 |
| | cold | func. | 1 | preserve_most | func. | 1 |
| | constructor | func. | 1 | preserve_none | func. | 1 |
| | cpu_specific | func. | 1 | regcall | var. | 1 |
| | ifunc | func. | 1 | sanitize | func. | 1 |
| | speculative | func. | 1 | target | func. | 1 |

"#" denotes the number of bugs detected by Atlas.

by specifying the `optnone` attribute on the function. Notably, attributes targeting vectorization are the most effective bug finders in both GCC and LLVM. The `vector_size` attribute, when applied to variables, is the single most successful attribute, contributing to 14 (10 + 4) bugs. Following closely is the `simd` attribute applied to statements, which is responsible for uncovering nine bugs in GCC. Function-level attributes also remain critical, with `return_twice` being linked to seven bugs. These findings highlight that bugs are not confined to one type of attribute but are spread across variable, statement, and function-level declarations. A variety of optimization-related attributes were also identified. Additionally, the attributes `always_inline`, `noipa`, and `noinline` are utilized to control function-level optimizations, influencing how specific functions are treated during the optimization process. These findings highlight the effectiveness of our approach in uncovering bugs by fine-tuning function optimization through attribute manipulation. The `vector_size` attribute, which can be

applied to both variables and functions, requires careful distinction based on its usage context. This attribute proves effective in detecting bugs in both GCC and LLVM, as compilers often struggle with optimizing code when `vector_size` is used, particularly in complex expressions or with multiple vectorized operations. Furthermore, the `simd` attribute uncovered several bugs in both compilers due to its role in generating SIMD instructions for loops. Automatically transforming scalar operations into vectorized forms is a complex process; the compiler must analyze dependencies and ensure that operations can be executed in parallel without introducing errors. Such observations suggest that compiler developers should pay more attention to these attributes to ensure the compiler's correctness.

*Commonly Used Attributes.* The attributes that triggered bugs found by Atlas are not rarely used; rather, they are commonly found in real-world software systems. For instance: the `optimize` and `optnone` attributes are used to enforce or disable optimization on specific functions, which is common in debugging, benchmarking, and embedded systems; The `simd` attribute is prevalent in **High-Performance Computing (HPC)**, multimedia processing, and numerical libraries where loop vectorization is critical; The `malloc` attribute is frequently applied to memory allocation routines to improve alias analysis and enable optimizations; The `aligned` attribute is widely used in performance-critical software to ensure proper memory alignment for data structures, particularly in low-level systems code and signal processing; The `vector_size` attribute is common in SIMD programming and is used in many numerical and graphics applications. These attributes appear regularly in system libraries, graphics engines, **Deep Learning (DL)** frameworks, HPC software, and performance-tuned applications. As a result, the bugs associated with them are likely to be triggered in real-world development.

### 4.3 RQ2: Atlas Efficiency

*Baseline and Comparative Approaches.* To comprehensively evaluate our methods, we compare our two proposed strategies, the SS and the LGS, against several key baselines. These comparisons are designed to isolate the contributions of attribute insertion, semantic guidance from the LLM, and compilation option selection.

- **Random Option (RO):** This approach serves as our most fundamental baseline. It involves compiling original, unmodified programs with randomly selected optimization flags, without any attribute insertion. By comparing both SS and LGS against RO, we can measure the core benefit of using an attribute-guided methodology over random compiler testing.
- **Attribute Only (AO):** To isolate the impact of our option selection mechanisms, we use an ablation study where attributes are inserted but compilation options are omitted. Comparing our full strategies against AO allows us to quantify how much a targeted selection of compilation options contributes to the effectiveness of the inserted attributes.
- *EMI:* Finally, we compare both SS and LGS against Hermes[54], a leading fuzzer based on the EMI. This comparison benchmarks our attribute-focused methods against a powerful differential testing approach.

This multi-faceted evaluation framework allows us to not only demonstrate the overall bug-finding effectiveness of SS and LGS but also to precisely discern the distinct advantages conferred by: (1) using attributes versus ROs; (2) the semantic intelligence of the LLM (LGS vs. SS); (3) the synergy of pairing attributes with compilation options (vs. AO). We use Csmith and YARPGen as the generators to generate the test programs. To evaluate the robustness of the Atlas-LGS framework and its dependency on a specific LLM, we extended our evaluation to include two additional models: Gemini 2.5 Flash [49] and Qwen Plus [50]. These models were selected specifically for their demonstrated capabilities in handling complex code reasoning tasks at the low latency and

high throughput required for large-scale fuzzing applications [24]. We denote these variants as Atlas-LGS-G (Gemini) and Atlas-LGS-Q (Qwen), respectively, while the original variant using DeepSeek V3 is referred to as Atlas-LGS-D. This evaluation allows us to highlight the efficacy of the Atlas-LGS framework itself. Specifically, we demonstrate how our framework effectively leverages the semantic reasoning of diverse LLMs to analyze code logic and insert suitable attributes.

*Measurements.* Following the existing work [6, 7, 62], we used the number of detected bugs within the same testing period to measure the performance of Atlas. Same as the existing study [6, 7, 62], we tested each subject using each approach for 10 days, and adopted Correcting Commits [4], a method commonly used in existing studies [6, 8, 62], to identify the number of detected bugs from a set of failing test programs. More specifically, for any test program that triggers a bug of a compiler $C$ whose commit version is $x$, the Correcting Commits method checks subsequent commits of the compiler and determines which commits fix the bug. If two failing test programs have the same correcting commit, they are regarded as triggering the same bug. The number of correcting commits is approximately regarded as the number of detected bugs. In fact, the accuracy of this method is quite substantial as existing studies [4, 8] demonstrate. This method may be a threat to our study. However, it is the only automatic method to measure the number of detected bugs and its accuracy has been demonstrated by the existing study, and thus, this threat may be not serious. We applied *RO*, *AO*, *EMI*, and Atlas to test GCC-5.1 and LLVM-5.0.0. We choose these old releases rather than the latest release since: (1) they are the stable versions that support sanitizers [33] and the compilation options includes sanitizer options; (2) they released for a long time, and thus, most bugs have been exposed; (3) most bugs have been fixed as they have been maintained for a long period, thus we will have enough correcting commits to evaluate different strategies.

*Number of Detected Bugs.* Table 4 shows the number of detected bugs within 10 days. Our analysis of these results reveals a clear tradeoff between Atlas-SS and Atlas-LGS variants. Our stochastic approach, Atlas-SS, proved to be the most effective in terms of bug counts. With Csmith, Atlas-SS identified 49 bugs, and with YARPGen, it found 38. This is a significant improvement over both RO (18 and 14 bugs) and AO (30 and 22 bugs), demonstrating that the synergy between inserting attributes and selecting their corresponding compilation options is crucial for effectively stressing the compiler and uncovering a large number of bugs.

Our expanded experiment comparing three different LLMs further highlights the robustness of the LGS framework. We observe that all three models show a comparably high level of performance. While there are minor variations, with Atlas-LGS-D showing a slight strength in efficiency (lowest programs-per-bug ratio), no single model dramatically outperforms the others. This consistency is a significant finding, as it suggests that the effectiveness of the Atlas-LGS framework is not rigidly dependent on a single LLM architecture. Instead, the underlying methodology of using LLMs for semantically aware attribute insertion is robust and adaptable across different models.

Our findings indicate that the core strength of Atlas-LGS lies in its strategic framework. This provides a promising outlook for future work, where the framework could potentially be paired with more cost-effective or specialized open source models as they evolve, further enhancing its practicality without compromising its high efficiency. A practical evaluation of the LGS must also account for the operational costs that lead to this lower throughput. During our 10-day experiment, we measured the monetary and time costs associated with the API calls for each of the three LLMs. DeepSeek V3 incurred a monetary expenditure of $67.20 USD with an average API latency of approximately 12 seconds; Gemini 2.5 Flash cost $79.30 USD with a latency of approximately 14 seconds; and Qwen Plus cost $68.60 USD with a latency of approximately 11 seconds. The key empirical finding from these data is that the operational costs and performance profiles are consistent across these different models. This analysis provides the view of the practical costs and

Table 4. The Number of Detected Bugs and Generated Compilable Programs within 10 Days

| Generators | Techniques | GCC Bugs(#) | LLVM Bugs(#) | Total Bugs(#) | Compilable Programs(#) | Total Programs(#) |
|---|---|---|---|---|---|---|
| Csmith | **RO** | 13 | 5 | 18 | 53,458 | 53,481 |
| | **AO** | 19 | 11 | 30 | 32,043 | 47,825 |
| | **EMI** | 17 | 11 | 28 | 48,765 | 49,279 |
| | **Atlas-SS** | 32 | 17 | 49 | 30,170 | 46,416 |
| | **Atlas-LGS-D** | 19 | 5 | 24 | 15,380 | 20,547 |
| | **Atlas-LGS-G** | 17 | 5 | 22 | 14,935 | 19,386 |
| | **Atlas-LGS-Q** | 16 | 6 | 22 | 15,967 | 20,385 |
| YARPGen | **RO** | 8 | 6 | 14 | 123,087 | 123,192 |
| | **AO** | 12 | 10 | 22 | 70,361 | 108,085 |
| | **EMI** | 10 | 9 | 19 | 101,734 | 103,628 |
| | **Atlas-SS** | 25 | 13 | 38 | 62,142 | 99,439 |
| | **Atlas-LGS-D** | 11 | 4 | 15 | 27,686 | 38,745 |
| | **Atlas-LGS-G** | 10 | 4 | 14 | 25,991 | 35,836 |
| | **Atlas-LGS-Q** | 13 | 3 | 16 | 28,169 | 39,572 |

strengthens the generalizability of our findings by demonstrating that the LGS framework is robust and not dependent on a specific model.

*Number of Unique Bugs.* To understand the unique contributions of each testing strategy, we visualize the overlap of discovered bugs in Figure 9. The diagrams clearly illustrate the distinct strengths of our proposed methods, Atlas-SS and Atlas-LGS, compared to the baselines. A key insight from the diagrams is the complementary nature of our two strategies. For programs generated by Csmith, Atlas-SS discovered eight unique bugs that no other method could, while Atlas-LGS identified its own distinct set of six unique bugs. This pattern is consistent with YARPGen, where *SS* and *LGS* uncovered six and four unique bugs, respectively. This demonstrates that the SS and the LLM-guided strategy explore different bugs in the compiler, making them highly complementary. Combining both strategies allows for a more comprehensive testing approach than either could achieve alone. Furthermore, the results highlight the critical role of pairing attributes with their corresponding compilation options. Notably, every bug discovered by the *AO* baseline was also found by either Atlas-SS or Atlas-LGS. This is powerful evidence that our option-selection mechanism is a strict improvement, as it preserves all of *AO*'s bug-finding capabilities while adding the ability to find many more bugs that *AO* misses.

While our combined approach is superior, it is worth noting that *EMI* still finds a small number of unique bugs (8 with Csmith, 6 with YARPGen). This is expected, as *EMI* employs a fundamentally different, structure-modifying differential testing approach. Additionally, as shown in Table 4, *EMI* tests a significantly larger number of programs than our methods. As prior work has shown [7], the number of test programs can influence bug discovery, which likely accounts for the unique bugs found by *EMI*. However, the fact that both Atlas-SS and Atlas-LGS find their own large sets of unique bugs underscores the novelty and effectiveness of our attribute-guided methodology.

To further investigate the consistency and unique contributions of each LLM, we analyze the overlap of the bugs discovered by the three LGS variants in Figure 10. The first finding is the high degree of overlap among the models. For Csmith, a substantial set of 20 bugs was found by all three LLMs, representing the vast majority of findings for each model. This pattern is mirrored

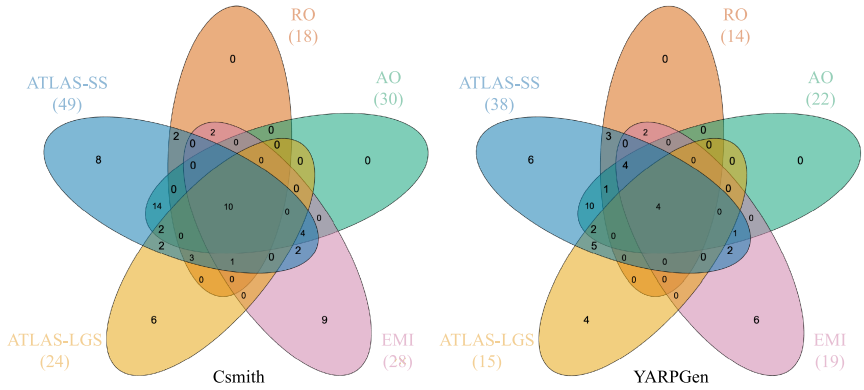



Fig. 9. Unique bugs found by different techniques.

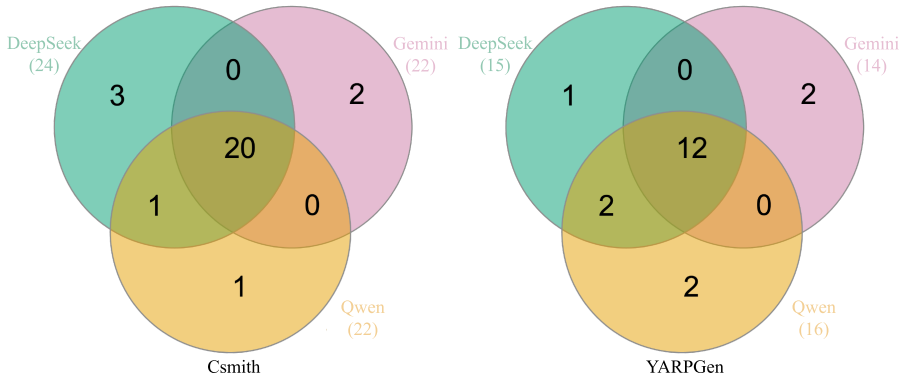



Fig. 10. Unique bugs found by different LLMs.

with YARPGen, where a common set of 12 bugs was identified by all variants. This significant intersection provides strong evidence for the robustness and reliability of the LGS framework itself. It demonstrates that the core methodology of using semantic guidance for attribute insertion leads different models to discover the same set of important, underlying bugs, rather than the success being an artifact of one particular model's strengths. However, the diagrams also reveal that each model makes unique contributions. For instance, with Csmith seeds, DeepSeek found three bugs that the other models missed, while Gemini found two and Qwen found one. The results show our LGS framework is solid, because all three LLMs found the most of bugs. The small number of unique bugs each one found is probably due to their different training data.

There is a different performance between LLVM and GCC in Table 4. We summarize the reasons as follows: (1) The release time of the LLVM subjects is about 2 years later than the GCC subjects. Thus, these LLVM subjects should have fewer bugs than those GCC subjects. The chosen LLVM subjects are LLVM-5.0.0 released on 7 September 2017 [43]. While GCC-5.1 were released in 22 April 2015 [21]. There is a 2-year gap between the LLVM subjects and the GCC subjects. In addition, Csmith was released in 2011. At that time, Csmith found 203 bugs for LLVM and 79 for GCC [65]. In other words, most bugs found by Csmith have been fixed in those LLVM subjects as they were released 2 years later after Csmith. (2) Each LLVM version is maintained for a shorter period than the three subject GCC versions. LLVM-5.0.0 was only maintained for 10 months [45], while GCC-5.1 are maintained for 30 months [19]. Since the GCC subjects were maintained for much longer than

the LLVM subjects, the GCC subjects should have much more bug-fixing commits than LLVM during the maintenance period.

*Unique Contributions of SS and LGS on New Bugs.* While our efficiency evaluation on older compilers demonstrated that LGS finds a unique set of bugs, a direct comparison on the latest compiler versions allows for a more comprehensive evaluation of its unique contributions. To this end, we conducted a follow-up experiment. To ensure our comparison was as rigorous as possible, we needed a ground truth set of confirmed, non-duplicate bugs. Of the new bugs in Table 1 initially discovered by LGS, 15 have been confirmed or fixed by the compiler developers. We used this set of 15 validated bugs as the basis for our new experiment, as their confirmation guarantees they are real and unique issues. We then deployed the SS on the exact same recent trunk revisions to determine how many of these 15 specific, validated bugs it could reproduce. As our primary method for distinguishing failures, we treat crashes with different stack traces as unique bugs. The results were definitive. Out of the 15 confirmed and non-duplicate bugs originally found by LGS, our SS implementation was able to reproduce only 3. This means that the remaining 12 bugs could not be found by the SS. These 12 bugs account for 80% of all the developer-validated discoveries from LGS in this comparison. A manual analysis of these 12 LGS-exclusive bugs revealed a consistent pattern: the majority are triggered by the application of complex pragmas, which require a semantic understanding of control-flow structures like loops. While SS can perform a basic structural analysis to apply simple pragmas, it lacks the deeper comprehension to construct the sophisticated and semantically relevant directives needed to trigger these specific bugs. This demonstrates that LGS explores corners of the compilation space that are inaccessible to SS, thereby uncovering a more valuable class of deep bugs.

*Compilable Programs.* As shown in Table 4, *RO* and *EMI* generated the highest proportion of compilable programs among these approaches. This outcome is expected, as the programs generated by Csmith and YARPGen strictly adhere to the C language standard, whereas *EMI* performs equivalent mutations on these programs. The total number of test programs generated by *AO* and Atlas is smaller than those produced by *RO* and *EMI*. This is because the program generation phase in *AO* and Atlas involves both modifying the original program and selecting attributes, resulting in slightly higher overhead compared to *EMI*. Additionally, the proportion of programs successfully compiled by *AO* and Atlas is lower than that of *RO* and *EMI*. This is primarily due to the insertion of certain attributes that can conflict with the program's logic. Attributes related to variables, such as `vector_size`, are a particular source of these conflicts, as they can violate the program's requirements for how a variable is used.

While LGS generates fewer test programs, an analysis of the compilable programs reveals its key strength: precision. The proportion of compilable programs generated by LGS is substantially higher than that of SS. For Csmith, the compilability rate of Atlas-LGS-D is approximately 75% (15,380/20,547), far exceeding the 65% rate of Atlas-SS (30,170/46,416). A similar trend is observed with YARPGen, where LGS-D achieves a 71% success rate compared to 62.5% for SS. This trend holds for all three LLMs and demonstrates that the semantic analysis provided by the LLMs is effective at proposing valid, context-appropriate attributes, thus reducing compilation failures. This higher success rate is attributed to the LLM's semantic analysis capabilities. Unlike the purely stochastic approach of SS, the LLM recommends attributes that are semantically appropriate for the specific code snippet under analysis. This intelligent selection process leads to fewer compilation failures caused by invalid attribute usage. This demonstrates the effectiveness of the LGS approach in generating valid and meaningful test cases. In summary, these results illustrate a clear tradeoff: Atlas-SS excels in high-throughput bug discovery through broad, random exploration, while Atlas-LGS offers a more precise, high-quality approach that, despite its lower throughput, generates more valid and semantically meaningful test cases per attempt.

*Time Trends.* Figure 11 illustrates the time trends in the number of unique bugs discovered for both GCC and Clang over a 10-day period. The top two subplots show the results for Csmith-generated programs, and the bottom two for YARPGen. A clear and consistent trend across all four scenarios is the high efficiency of our SS, Atlas-SS. It exhibits the steepest discovery curve from the outset and maintains a sustained rate of finding new bugs through the entire testing period, showing no signs of saturation. In contrast, the Atlas-LGS variants (orange, yellow, and brown lines) discover a lower absolute number of bugs within the same period. This is not an indication of a weaker strategy but is an natural consequence of its design. The lower throughput of Atlas-LGS is a direct result of the time-intensive nature of making API calls to the LLMs for semantic analysis. Despite this overhead, the plots show that the LGS variants also maintain a steady, continuous ability to find new bugs throughout the testing period, without showing signs of saturation. Crucially, the comparison between the LGS variants reveals their consistency. The lines for LGS-D, LGS-G, and LGS-Q are tightly clustered in all four plots, indicating that they have a very similar bug discovery rate and overall effectiveness. This is a powerful finding, as it demonstrates the robustness of the LGS framework itself. The success of the semantic-aware strategy is not an artifact of a single model but is a generalizable property of the methodology. When compared to the baselines, both of our strategies show clear advantages. The *RO* approach (blue line) tends to saturate relatively quickly, especially with Csmith. While *AO* and *EMI* perform better than *RO*, they are consistently outperformed by Atlas-SS. The superior performance of both the *AO* and Atlas-SS strategies over *RO* and *EMI* demonstrates that an attribute-guided method is an efficient way to discover compiler bugs over time. The LGS variants, despite their lower throughput, remain competitive with the AO and EMI baselines, reinforcing their value as effective bug-finding tools.

### 4.4 RQ3: Code Coverage

To evaluate how effectively our methods explore the compiler's internal logic, we conducted two complementary code coverage experiments, each designed to answer a different question about performance. The first experiment, based on a fixed number of seed programs, measures the inherent potency of individual test cases. The second, based on a fixed time budget, measures the overall practical effectiveness of each strategy over a sustained period. The compilation time of each program was constrained to a maximum of 10 seconds, and we measured the code coverage achieved during these compilations.

*Fixed Number of Seed Programs.* Our first experiment isolates the quality of the generated test cases. Following the methodology of previous work [34], we randomly generated 10,000 seed programs using Csmith. These test programs were distinct from those generated in Section 4.3. For each of our five techniques (RO, AO, EMI, ATLAS-SS, and the original ATLAS-LGS-D), we used these seeds to test the latest compiler releases, GCC-15.0.0 and LLVM-19.0.0. By giving each strategy the same number of seed programs, this experiment directly measures which strategy produces individually more effective programs. The results are summarized in Table 5 (the best results are marked in gray). The data clearly show that both our proposed strategies, Atlas-SS and Atlas-LGS, significantly outperform all baseline approaches (RO, AO, and EMI) across every measured metric for both GCC and LLVM. Although the coverage percentages are relatively similar, a detailed analysis of the specific values reveals an improvement of tens of thousands in the number of covered lines and branches.

Most notably, Atlas-LGS achieved the highest coverage in all categories. Since each technique started with the same 10,000 seed programs, this superior coverage is direct evidence that Atlas-LGS generates higher-quality test cases. By leveraging the LLM's semantic analysis, this strategy places attributes in a more meaningful and context-aware manner, allowing it to exercise more complex and unique code paths within the compiler that are missed by other methods. These results

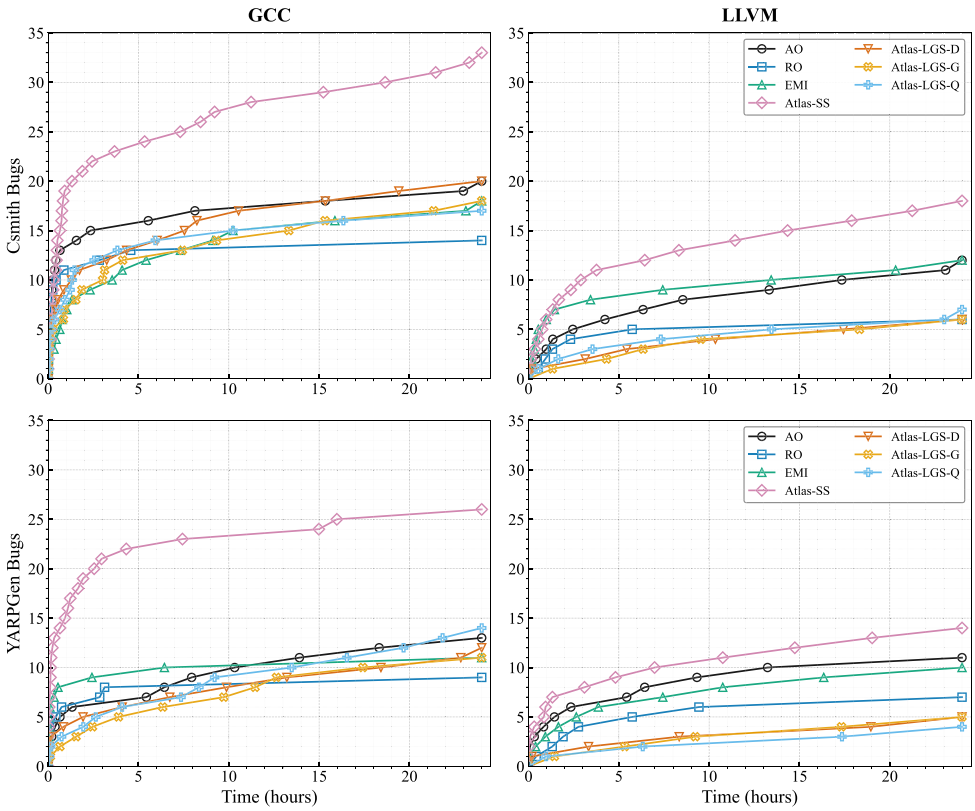



Fig. 11. The earliest time each unique bug was discovered.

Table 5. Code Coverage of GCC and LLVM on a Fixed Number of Test Programs

| Compiler | Techniques | Line | Function | Branch |
|---|---|---|---|---|
| GCC | **RO** | 33.5% | 35.2% | 21.1% |
| | **AO** | 34.7%(+13,080) | 36.9%(+419) | 22.0%(+10,697) |
| | **EMI** | 35.3%(+19,619) | 36.8%(+314) | 22.4%(15,450) |
| | **Atlas-SS** | 36.3%(+30,519) | 37.6%(+1,152) | 23.2%(+24,958) |
| | **Atlas-LGS** | 36.5%(+32,954) | 37.7%(+1,203) | 23.3%(+26,243) |
| LLVM | **RO** | 34.4% | 25.6% | 20.0% |
| | **AO** | 35.6%(+20,324) | 26.5%(+849) | 20.9%(+6,472) |
| | **EMI** | 35.1%(+12,435) | 26.9%(+1,307) | 21.3%(+9,592) |
| | **Atlas-SS** | 36.8%(+41,806) | 27.6%(+1,953) | 21.7%(+12,954) |
| | **Atlas-LGS** | 37.4%(+52,783) | 28.1%(+2,475) | 22.2%(+16,278) |

validate the effectiveness of our attribute-guided approach. Both the SS and the LGS enhance the thoroughness of compiler testing by triggering more code paths, thereby increasing the potential for uncovering hidden bugs and demonstrating the practical value of our work.

*Fixed Time Budget.* The second experiment addresses the question of practical effectiveness, accounting for the different throughputs of each strategy. We tested each of our strategies, including

Table 6. Code Coverage of GCC and LLVM over a 10-Day Period

| Compiler | Techniques | Line | Function | Branch |
|---|---|---|---|---|
| GCC | **RO** | 33.6% | 35.3% | 21.1% |
| | **AO** | 34.5%(+11,974) | 36.6%(+221) | 21.9%(+9,053) |
| | **EMI** | 35.1%(+17,043) | 36.6%(+213) | 22.2%(13,974) |
| | **Atlas-SS** | 36.1%(+27,341) | 37.3%(+955) | 23.1%(+22,178) |
| | **Atlas-LGS-D** | 36.3%(+29,563) | 37.5%(+1,041) | 23.2%(+24,118) |
| | **Atlas-LGS-G** | 36.2%(+28,035) | 37.4%(+998) | 23.2%(+23,953) |
| | **Atlas-LGS-Q** | 36.3%(+29,397) | 37.4%(+1,028) | 23.2%(+24,643) |
| LLVM | **RO** | 34.6% | 25.7% | 20.1% |
| | **AO** | 35.3%(+13,574) | 26.3%(+617) | 20.7%(+4,197) |
| | **EMI** | 35.7%(+17,354) | 26.8%(+1,175) | 21.2%(+7,629) |
| | **Atlas-SS** | 36.6%(+33,753) | 27.4%(+1,693) | 21.6%(+10,845) |
| | **Atlas-LGS-D** | 37.2%(+43,068) | 27.9%(+2,179) | 22.1%(+13,586) |
| | **Atlas-LGS-G** | 37.1%(+41,977) | 27.6%(+2,083) | 22.0%(+12,356) |
| | **Atlas-LGS-Q** | 37.1%(+42,462) | 27.7%(+2,136) | 22.0%(+12,864) |

The shading indicates the best-performing technique.

the three Atlas-LGS variants (LGS-D, LGS-G, and LGS-Q), for a continuous 10-day period on the latest compiler releases and measured the coverage achieved. The results of this time-budgeted evaluation are summarized in Table 6. As expected, the RO achieved the lowest coverage because it lacks targeted guidance. Both AO and EMI significantly outperformed RO, which demonstrates that manipulating program characteristics is more effective than simply varying compilation flags. However, the Atlas-SS achieved a substantial improvement over both AO and EMI. This highlights a key advantage of our framework: Atlas-SS not only inserts attributes but also pairs them with the specific compilation options required to activate them. This creates complex compilation scenarios that force the compiler to handle intricate interactions between attributes and optimizations, leading to significantly higher coverage of its internal logic.

Despite the lower throughput of Atlas-LGS due to LLM API latency, all LGS models achieve higher code coverage than the high-volume Atlas-SS. This provides powerful evidence for a "quality over quantity" effect. While SS is highly effective at a broad, syntactically driven exploration, LGS operates on a deeper, semantic level. It generates test cases that are more targeted and contextually relevant, allowing each individual program to probe more complex and harder-to-reach corner cases within the compiler's analysis and optimization passes. This higher-quality exploration results in greater coverage over a fixed time budget.

A comparison of the different LLM-driven variants reveals consistent performance. While Atlas-LGS-D shows a slight lead, the results for DeepSeek, Gemini, and Qwen are very similar. This finding demonstrates the robustness of the Atlas-LGS framework. The effectiveness of our approach relies on the methodology of using semantic guidance rather than the specific capabilities of a single model. The minor differences observed are likely due to variations in the training data of each model, which may give one model a slight advantage in understanding specific language features.

In summary, our time-budgeted analysis confirms that attribute-guided methods are superior for exploring compiler internals. Furthermore, it demonstrates that the semantic intelligence of Atlas-LGS produces high-quality test cases that achieve the most thorough code exploration. This validates Atlas-LGS as a powerful and complementary strategy for deep compiler testing.

# 5 Discussion

## 5.1 Threats to Validity

We have identified the following main threats to the validity of our study.

*5.1.1 External Validity.* This concerns the generalizability of our findings.

*Representativeness of Subjects.* A primary threat is whether the chosen compilers and test programs are representative of real-world scenarios. To mitigate this, we selected GCC and LLVM, the two most dominant C compilers, which is a standard practice in compiler testing research [6, 65]. Furthermore, we evaluated multiple versions of each compiler to ensure our findings are not specific to a single release. For test program generation, we used Csmith and YARPGen, the most widely adopted tools in the field [17, 29]. While our results are promising, their applicability to other compilers (e.g., ICC, MSVC) remains an area for future work.

*Generalizability of the LGS.* A potential threat to our LGS is its dependence on a particular LLM. The performance and suggestions of an LLM are tied to its specific architecture and training data. To substantially mitigate this threat, we expanded our evaluation to include three distinct LLMs: DeepSeek V3, Gemini 2.5 Flash, and Qwen Plus. Our findings (Table 4) indicate that all three models delivered a comparably high level of performance in terms of bug-finding efficiency, suggesting that the effectiveness of our approach stems from the LGS framework itself, rather than the specifics of a single model. This demonstrates the robustness of our methodology. However, while this mitigates the threat for powerful, general-purpose models, generalizability to smaller, more specialized, or open source LLMs remains an important area for future investigation.

*5.1.2 Internal Validity.* This concerns the causal claims within our experiment.

*Influence of Prompt Engineering.* The effectiveness and consistency of the LGS are highly dependent on the quality of the prompt used to query the LLM. To mitigate this threat, we implemented the following strategy. First, to ensure the output is as deterministic and reproducible as possible, we use a `temperature=0` setting in all API calls. This instructs the model to always select the most probable next token, thereby minimizing randomness. Second, our prompt template (Figure 7) is carefully engineered to guide the model through a consistent reasoning framework for every query. By assigning it a specific role ("compiler fuzzing expert") and providing clear, sequential directives, we promote process consistency in its analysis. While this approach makes our methodology robust, it is possible that alternative prompt designs could lead to different or even better results. Our study demonstrates the viability of our specific prompt-based approach but does not claim it is the single optimal design.

*Completeness of the Attribute Set.* For our SS, we manually curated a set of attributes from the GCC and LLVM documentation. While comprehensive, this set may not be exhaustive. Similarly, the LGS is limited by the LLM's knowledge, which is derived from its training data. It is possible that both strategies might miss opportunities to use newer or more obscure attributes that could trigger additional bugs.

*5.1.3 Construct Validity.* This concerns how well our metrics measure what we claim to measure.

*Accuracy of Bug Counts.* Our primary metric for bug-finding effectiveness relies on counting unique bugs based on their "correcting commit," a standard and widely accepted automated method in recent studies [4, 6]. While this is the most feasible approach for a large-scale study, it is an estimation and may not perfectly map to the number of distinct root causes. To increase confidence, we manually analyzed crash messages to filter out obvious duplicates, and we found false positives to be rare.

*Code Coverage as a Proxy for Effectiveness.* We use code coverage to measure how thoroughly our methods explore the compiler. While higher coverage is a strong indicator that new code paths are

being exercised, it is not a perfect proxy for bug-finding ability. It is possible to increase coverage in non-buggy parts of the compiler's code. However, we believe that the consistent and significant increase in coverage provided by our Atlas-LGS and Atlas-SS strategies, combined with their superior bug-finding results, provides strong evidence for their effectiveness.

### 5.2 Cost Analysis and Tradeoffs of the LGS

The core viewpoint of our argument is that the value of LGS lies not in the *quantity* of test cases it generates, but in their superior *quality*, uniqueness, and depth, which allows it to uncover bugs that other methods cannot. We elaborate on this tradeoff through the following points:

*Discovery of Unique Bugs.* The strongest evidence for the necessity of LGS is its demonstrated ability to find exclusive bugs. As shown in our head-to-head comparison on the latest compiler versions (Section 4.2), the SS was unable to find 12 of the 15 (80%) new bugs that were discovered by LGS and confirmed or fixed by developers. Our analysis revealed that these bugs were tied to complex, semantic interactions that a stochastic approach is highly unlikely to generate. This indicates that LGS is not merely an alternative to SS, but a necessary complement for achieving deeper testing.

*Deeper Exploration of Compiler Code Paths.* Our code coverage analysis (Table 6) provides further validation for the high-quality nature of LGS-generated tests. Despite generating far fewer test cases, LGS consistently achieved the highest code coverage across all metrics. This indicates that its semantically guided test cases are more effective, enabling them to exercise complex and under-tested code paths within the compiler that other methods fail to reach.

*A Complementary Strategy.* The LGS is not intended to be a wholesale replacement for the low-cost, high-throughput SS. Rather, it serves as a powerful complementary tool. An optimal and comprehensive compiler testing methodology can leverage both strategies synergistically: using SS to find a broad range of bugs rapidly, and deploying LGS as a high-precision instrument to uncover deep, hard-to-find bugs. This hybrid approach maximizes both the breadth and depth of bug discovery, justifying the specialized role and associated cost of LGS.

### 5.3 Comparison with Rule-Based and Static Analysis Approaches

A crucial design choice in our LGS is the use of an LLM for semantic analysis. It is valuable to compare this approach with a more traditional static analysis or rule-based system for the same task of generating meaningful attributes. A well-designed static analysis system would be highly efficient at filtering out invalid attribute insertions; however, we argue that the LLM provides distinct advantages in creativity, deep contextual analysis, and adaptability that are challenging to replicate with predefined rules. First, a key distinction lies in the capacity for creative synthesis. A rule-based system is fundamentally bound by the heuristics and patterns its designers can foresee and encode. It excels at applying known good practices. In contrast, the LLM, trained on a vast and diverse corpus of real-world code, can identify and synthesize "creative" combinations of features that might not be captured by any single, predefined rule. It can suggest unforeseen interactions between attributes and specific code structures that might never be attempted by a system limited to prescriptive logic. This capacity for creative synthesis is a key mechanism for discovering novel bugs that arise from unexpected feature interactions. Second, the LLM provides a deeper level of contextual analysis. Static analysis tools are often excellent at identifying structural patterns, such as a loop amenable to vectorization. However, in complex functions with intricate control flow and data dependencies, the LLM's ability to develop a more holistic understanding of a function's semantic purpose becomes a distinct advantage. It can analyze the interplay between different parts of a function and suggest attributes relevant to the function's overall behavior, not just a single syntactic feature, enabling more sophisticated stress tests of the compiler. Finally, from a practical

perspective, the LLM offers significant advantages in terms of adaptability and maintenance. A static rule set requires continuous, manual updates by domain experts to keep pace with new compiler attributes, pragmas, or semantic changes. This creates a significant maintenance burden. An LLM-based framework is inherently more scalable. As future versions of the model are trained on newer data, they will automatically learn emerging attributes and their usage patterns, reducing the need for manual intervention. In summary, the role of the LLM is not to replace static analysis but to complement it. It acts as an intelligent guidance system, nudging the fuzzer toward a more promising and semantically richer set of candidate programs than a purely random or simple rule-based system could generate, thereby justifying its inclusion in our framework.

### 5.4 Scope of Bug Detection and Future Directions

To date, Atlas's findings have primarily consisted of ICEs rather than miscompilations. This outcome is a direct reflection of our current strategy's design, which prioritizes a broad exploration of the compilation space by leveraging a wide and diverse set of attributes. This approach has shown highly effective at uncovering bugs in the compiler. We acknowledge that finding miscompilations often requires a more targeted strategy that focuses on a specific class of attributes known to stress the compiler's semantic correctness. For instance, systematically using `__attribute__((optimize(''.'')))` and `__attribute__((optnone))` could reveal bugs where a function behaves differently when optimized in isolation. Our current implementation does not give special weight to these specific attributes. Looking forward, we believe that our Atlas-LGS framework is fundamentally better suited for this task. While Atlas-SS can randomly apply these potent attributes, it does so without context. In contrast, the semantic understanding of LGS could allow it to construct a particularly challenging test case. This points to an exciting avenue for future work: configuring Atlas with a "miscompilation-finding mode" that leverages LGS to prioritize these optimization-related attributes. This would be a natural extension of our framework and an excellent way to probe for compiler soundness issues.

### 5.5 Integration with Other Techniques

Atlas can be easily integrated with other compiler testing techniques. Specifically, Atlas can take test programs generated by tools such as Csmith [65], YARPGen [37, 38], and EMI [29] as seed inputs. For each seed program, Atlas parses its AST and identifies locations where attributes can be meaningfully inserted. By systematically injecting attributes at these locations, Atlas enables exploration of additional compiler compilation space, such as those involving optimization, code generation for special attributes, or attribute-sensitive transformations. Take EMI [29] as an example. EMI generates program variants that are semantically equivalent on a given input. After these variants are created, Atlas can further process each of them by analyzing their ASTs and inserting attributes at appropriate positions. This layered approach enables the discovery of bugs that only manifest under specific combinations of program structure and attributes. A similar integration can be achieved with Creal [34], which generates test cases by inserting real-world functions. Atlas can apply attribute insertion to these programs as well. In summary, Atlas is highly compatible with existing compiler testing frameworks and can be seamlessly integrated to explore a broader compilation space.

## 6 Related Work

This section introduces the related work on compiler testing and CSE.

### 6.1 Compiler Testing Techniques

The identification of compiler bugs has been the subject of extensive research. Existing compiler testing approaches can be broadly categorized into generation-based, mutation-based, and LLM-based techniques, while significant research also targets specific dimensions of compiler reliability.

*Generation-Based Testing.* Considerable efforts have been dedicated to developing automated program generators that construct test cases from scratch. Csmith [65] is a widely used program generator specifically designed for C, which has successfully uncovered numerous compiler crashes and correctness bugs. It is worth noting that programs generated by Csmith are guaranteed to be devoid of undefined behavior. Inspired by Csmith, CLsmith [36] targets OpenCL compilers, while Morrisset et al. [46] extended Csmith to detect concurrency bugs by supporting mutexes and atomic variables. More recently, YARPGen [37, 38] was proposed to specifically focuses on testing scalar loop optimizations in C/C++ compilers. SPE focuses on enumerating variable usage patterns in skeleton-based test synthesis [68]. However, as noted in recent studies, rule-driven generators may eventually reach a saturation point where they struggle to uncover new bugs due to fixed generation constraints [2].

While generation-based tools like Csmith [65] and YARPGen [37] excel at constructing diverse program structures, they primarily rely on global compilation options to steer the compiler's behavior. These global options treat the entire program uniformly, limiting the ability to stress-test-specific code segments with conflicting or specialized constraints. In contrast, Atlas introduces a finer granularity of control by inserting attributes into individual program elements. This allows Atlas to enforce localized optimization strategies (e.g., forcing a specific function to be inlined while others are not) that are unattainable through global flags alone, thereby exploring a complementary dimension of the compilation space.

*Mutation-Based Testing.* Instead of generating programs from scratch, another line of research mutates existing seed programs. EMI [29] has been proposed for mutating or transforming seed programs while preserving their semantics under the same input. EMI can be implemented through techniques such as deleting dead statements [29], inserting new code in inactive regions [30], or synthesizing equivalent code in active regions [54]. When combined with Csmith, EMI has proven effective in uncovering thousands of compiler optimization bugs. GrayC designs five semantic-aware mutators specifically for creating more compilable programs [17]. To address C++ complexities, TyMut[59] employs type-driven mutations to target templates and inheritance. Complementing this, Wang et al. [58] analyzed 788 OOP-related bugs, revealing that object construction and destruction are particularly error-prone. Guided by these insights, they developed OOPFuzz to specifically target these deep-seated bugs. Beyond semantics-preserving mutations, tools like LangFuzz [25] and classfuzz [10] perform grammar-based substitutions or non-semantic mutations to test JavaScript interpreters and JVMs, respectively. Li et al. [34] use real-world code snippets to construct semantics-preserving programs for testing optimizing compilers. Nagai et al. [47] proposed a generator for random arithmetic expressions using non-semantics-preserving mutations to explore diverse expressions.

Mutation-based approaches like EMI [29] and GrayC [17] focus on altering program logic or structure to preserve semantics. However, they typically overlook the rich semantic information conveyed by programming language attributes. Atlas diverges from these methods by leveraging attributes as a distinct mutation operator. Instead of merely changing what the code computes, Atlas changes how the compiler perceives and processes specific code blocks (e.g., alignment directives or sanitization checks). This attribute-guided method enables Atlas to trigger obscure compiler paths that traditional structural mutations seldom reach.

*LLM-Based Testing*. Recent researchers have leveraged LLMs in compiler testing. Fuzz4All [63] represents a universal fuzzing approach, utilizing LLMs as both the input generator and the mutation engine for various targets. WhiteFox [64] integrates LLMs with source-code analysis to perform white-box testing specifically for DL compiler optimizations. Ou et al. [51] proposed using LLMs to generate mutators, and Gao et al. [18] introduced a bracket-based masking strategy driven by historical Rust compiler bugs to synthesize valid test inputs, leading to the discovery of multiple confirmed bugs. Ni and Li [48] propose LegoFuzz, which decouples testing into an offline phase where LLMs generate feature-rich code snippets, and an online phase that efficiently interleaves these snippets to synthesize complex and valid test programs.

Recent LLM-based fuzzers [48, 63] utilize LLMs to generate code from scratch or mutate existing code. While effective, these approaches often struggle with the "validity vs. diversity" tradeoff, as generating complex code that is also compilable and semantically meaningful is challenging. Atlas uniquely integrates LLMs not just for code generation, but for semantic-aware attribute injection (via our LGS strategy). By using LLMs to understand the context of existing code and recommend compatible attributes, Atlas bridges the gap between random fuzzing and semantic correctness, ensuring that the inserted attributes effectively stress the compiler without trivially breaking compilation.

*Other Aspects*. The scope of compiler testing extends beyond code generation errors to cover the reliability of the broader toolchain and code quality. Consequently, other aspects of compiler reliability, such as incorrect debug information [35, 60], debugger toolchain bugs [67], and missed optimizations [56, 57], have also been extensively studied.

### 6.2 CSE

Li et al. [32] introduced the concept of the CSE, which encompasses a comprehensive collection of **Just-In-Time (JIT)** compilation options. These options can be cross-validated to ensure the correctness of JIT compilation. To systematically explore the compilation space, Li et al. employed strategic mutations in the code structure of test programs. These mutations were thoughtfully designed to preserve the code's semantics while effectively triggering a wide range of JIT compilation choices. JOpFuzzer [28] focuses on JVM JIT compilers by navigating the optimization option space. It leverages learned correlations between program features and compiler flags to steer the fuzzing process toward configurations most likely to trigger bugs. Chen and Suo [6] conducted an empirical study on GCC and LLVM, highlighting the distinctive value of exploring the space of compiler optimizations. They proposed the COTest by considering both optimization settings and test cases to boost compiler testing.

Different from these approaches, our approach integrates not only the optimization options but also compilation options with the attributes to facilitate the exploration of the compilation space. Additionally, we introduce the insertion of attributes into the program, enabling a fine-grained and in-depth exploration of the compilation space. This integration of attributes with compilation options represents a novel approach, providing a more detailed and comprehensive examination of compiler behavior and reliability.

## 7 Conclusion

This study introduces a novel approach that utilizes attributes to guide the exploration of compilation space for compiler testing. To achieve this, we strategically incorporate attributes into existing test programs to generate mutated versions. Since attributes enable precise control over the compilation strategies applied to individual code segments, the mutated versions can be utilized to explore previously unexamined areas of compilers. We have implemented this concept as a prototype called Atlas. Through extensive evaluation, we demonstrate the practical value of Atlas, as it

successfully detected a total of 97 bugs for GCC and LLVM. Notably, 73 of these bugs have been confirmed, and 19 have been resolved by the developers, underscoring the significant impact and efficacy of Atlas in identifying compiler bugs.

## Acknowledgments

We thank the anonymous reviewers and editors for their constructive feedback and careful guidance, which substantially improved the quality and clarity of this work. We are also grateful to the GCC and LLVM developers, particularly Jakub Jelinek, Richard Biener, Marek Polacek, and Martin Jambor, for inspecting and fixing our reported bugs.

## References

[1] 2024. Attribute Specifier Sequence. Retrieved October 20, 2024 from https://en.cppreference.com/w/cpp/language/attributes

[2] Domenico Amalfitano, Nicola Amatucci, Anna Rita Fasolino, Porfirio Tramontana, Emily Kowalczyk, and Atif M. Memon. 2015. Exploiting the saturation effect in automatic random testing of android applications. In *Proceedings of the 2nd ACM International Conference on Mobile Software Engineering and Systems (MOBILESoft '15)*. IEEE Computer Society, 33–43. DOI: https://doi.org/10.1109/MOBILESOFT.2015.11

[3] Andrew W. Appel. 1998. *Modern Compiler Implementation in C*. Cambridge University Press.

[4] Junjie Chen, Wenxiang Hu, Dan Hao, Yingfei Xiong, Hongyu Zhang, Lu Zhang, and Bing Xie. 2016. An empirical comparison of compiler testing techniques. In *Proceedings of the 38th International Conference on Software Engineering (ICSE '16)*. ACM, 180–190. DOI: https://doi.org/10.1145/2884781.2884878

[5] Junjie Chen, Jibesh Patra, Michael Pradel, Yingfei Xiong, Hongyu Zhang, Dan Hao, and Lu Zhang. 2020. A survey of compiler testing. *ACM Comput. Surv*. 53, 1 (2020), Article 4, 1–36. DOI: https://doi.org/10.1145/3363562

[6] Junjie Chen and Chenyao Suo. 2022. Boosting compiler testing via compiler optimization exploration. *ACM Trans. Softw. Eng. Methodol*. 31, 4 (2022), Article 72, 1–33. DOI: https://doi.org/10.1145/3508362

[7] Junjie Chen, Guancheng Wang, Dan Hao, Yingfei Xiong, Hongyu Zhang, and Lu Zhang. 2019. History-guided configuration diversification for compiler test-program generation. In *Proceedings of the 34th IEEE/ACM International Conference on Automated Software Engineering (ASE '19)*. IEEE, 305–316. DOI: https://doi.org/10.1109/ASE.2019.00037

[8] Junjie Chen, Guancheng Wang, Dan Hao, Yingfei Xiong, Hongyu Zhang, Lu Zhang, and Bing Xie. 2021. Coverage prediction for accelerating compiler testing. *IEEE Trans. Softw. Eng*. 47, 2 (2021), 261–278. DOI: https://doi.org/10.1109/TSE.2018.2889771

[9] Tsong Yueh Chen, Shing-Chi Cheung, and Siu-Ming Yiu. 2020. Metamorphic testing: A new approach for generating next test cases. arXiv:2002.12543. Retrieved from https://arxiv.org/abs/2002.12543

[10] Yuting Chen, Ting Su, Chengnian Sun, Zhendong Su, and Jianjun Zhao. 2016. Coverage-directed differential testing of JVM implementations. In *Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '16)*. ACM, 85–99. DOI: https://doi.org/10.1145/2908080.2908095

[11] DeepSeek-AI. 2025. The Temperature Parameter of DeepSeek. Retrieved July 20, 2025 from https://api-docs.deepseek.com/quick_start/parameter_settings

[12] DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, et al. 2024. DeepSeek-V3 technical report. arXiv:2412.19437. Retrieved from https://arxiv.org/abs/2412.19437

[13] GCC Developers. 2024. Declaring Attributes of Functions. Retrieved October 20, 2024 from https://gcc.gnu.org/onlinedocs/gcc/Function-Attributes.html

[14] GCC Developers. 2024. Specifying Attributes of Types. Retrieved October 20, 2024 from https://gcc.gnu.org/onlinedocs/gcc/Type-Attributes.html

[15] GCC Developers. 2024. Specifying Attributes of Variables. Retrieved October 20, 2024 from https://gcc.gnu.org/onlinedocs/gcc/Variable-Attributes.html

[16] LLVM Developers. 2024. LLVM Function Attributes. Retrieved October 20, 2024 from https://clang.llvm.org/docs/AttributeReference.html

[17] Karine Even-Mendoza, Arindam Sharma, Alastair F. Donaldson, and Cristian Cadar. 2023. GrayC: Greybox fuzzing of compilers and analysers for C. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA '23)*. ACM, 1219–1231. DOI: https://doi.org/10.1145/3597926.3598130

[18] Hongyan Gao, Yibiao Yang, Maolin Sun, Jiangchang Wu, Yuming Zhou, and Baowen Xu. 2025. ClozeMaster: Fuzzing rust compiler by harnessing LLMs for infilling masked real programs. In *Proceedings of the 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE '25)*. IEEE/ACM, 712–712. DOI: https://doi.org/10.1109/ICSE55347.2025.00175

[19] GCC Developers. 2024. GCC Development Plan. Retrieved October 20, 2024 from https://gcc.gnu.org/develop.html

[20] GCC Developers. 2024. GCC Function Attributes. Retrieved October 20, 2024 from https://gcc.gnu.org/onlinedocs/gcc/Common-Function-Attributes.html

[21] GCC Developers. 2024. GCC Releases. Retrieved October 20, 2024 from https://gcc.gnu.org/releases.html

[22] GCC Developers. 2024. Installing GCC: Testing. Retrieved October 20, 2024 from https://gcc.gnu.org/install/test.html

[23] GCC Developers. 2024. Options That Control Optimization. Retrieved October 20, 2024 from https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html

[24] Google. 2025. Gemini API Documentation. Retrieved October 20, 2024 from https://ai.google.dev/gemini-api/docs/models

[25] Christian Holler, Kim Herzig, and Andreas Zeller. 2012. Fuzzing with code fragments. In *Proceedings of the 21th USENIX Security Symposium*. USENIX Association, 445–458. Retrieved from https://www.usenix.org/conference/usenixsecurity12/technical-sessions/presentation/holler

[26] Kenneth Hoste and Lieven Eeckhout. 2008. Cole: Compiler optimization level exploration. In *Proceedings of the 6th International Symposium on Code Generation and Optimization (CGO '08)*. ACM, 165–174. DOI: https://doi.org/10.1145/1356058.1356080

[27] Raphael Isemann, Cristiano Giuffrida, Herbert Bos, Erik van der Kouwe, and Klaus von Gleissenthall. 2023. Don't look UB: Exposing sanitizer-eliding compiler optimizations. *Proc. ACM Program. Lang.* 7, PLDI (2023), 907–927. DOI: https://doi.org/10.1145/3591257

[28] Haoxiang Jia, Ming Wen, Zifan Xie, Xiaochen Guo, Rongxin Wu, Maolin Sun, Kang Chen, and Hai Jin. 2023. Detecting JVM JIT compiler bugs via exploring two-dimensional input spaces. In *Proceedings of the 45th IEEE/ACM International Conference on Software Engineering (ICSE '23)*. IEEE, 43–55. DOI: https://doi.org/10.1109/ICSE48619.2023.00016

[29] Vu Le, Mehrdad Afshari, and Zhendong Su. 2014. Compiler validation via equivalence modulo inputs. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '14)*. ACM, 216–226. DOI: https://doi.org/10.1145/2594291.2594334

[30] Vu Le, Chengnian Sun, and Zhendong Su. 2015. Finding deep compiler bugs via guided stochastic program mutation. In *Proceedings of the 2015 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA '15)*. ACM, 386–399. DOI: https://doi.org/10.1145/2814270.2814319

[31] Vu Le, Chengnian Sun, and Zhendong Su. 2015. Randomized stress-testing of link-time optimizers. In *Proceedings of the 2015 International Symposium on Software Testing and Analysis (ISSTA '15)*. ACM, 327–337. DOI: https://doi.org/10.1145/2771783.2771785

[32] Cong Li, Yanyan Jiang, Chang Xu, and Zhendong Su. 2023. Validating JIT compilers via compilation space exploration. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*. ACM, 66–79. DOI: https://doi.org/10.1145/3600006.3613140

[33] Shaohua Li and Zhendong Su. 2024. UBFuzz: Finding bugs in sanitizer implementations. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1 (ASPLOS '24)*. ACM, 435–449. DOI: https://doi.org/10.1145/3617232.3624874

[34] Shaohua Li, Theodoros Theodoridis, and Zhendong Su. 2024. Boosting compiler testing by injecting real-world code. *Proc. ACM Program. Lang.* 8, PLDI (2024), 223–245. DOI: https://doi.org/10.1145/3656386

[35] Yuanbo Li, Shuo Ding, Qirun Zhang, and Davide Italiano. 2020. Debug information validation for optimized code. In *Proceedings of the 41st ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI '20)*. ACM, 1052–1065. DOI: https://doi.org/10.1145/3385412.3386020

[36] Christopher Lidbury, Andrei Lascu, Nathan Chong, and Alastair F. Donaldson. 2015. Many-core compiler fuzzing. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation*. ACM, 65–76. DOI: https://doi.org/10.1145/2737924.2737986

[37] Vsevolod Livinskii, Dmitry Babokin, and John Regehr. 2020. Random testing for C and C++ compilers with YARPGen. *Proc. ACM Program. Lang.* 4, OOPSLA (2020), Article 196, 1–25. DOI: https://doi.org/10.1145/3428264

[38] Vsevolod Livinskii, Dmitry Babokin, and John Regehr. 2023. Fuzzing loop optimizations in compilers for C++ and data-parallel languages. *Proc. ACM Program. Lang.* 7, PLDI (2023), 1826–1847. DOI: https://doi.org/10.1145/3591295

[39] LLVM Developers. 2024. Core Pipelines. Retrieved October 20, 2024 from https://llvm.org/docs/GlobalISel/Pipeline.html

[40] LLVm Developers. 2024. Installing GCC: Testing. Retrieved October 20, 2024 from https://llvm.org/docs/TestSuiteGuide.html
[41] LLVM Developers. 2024. LibTooling. Retrieved October 20, 2024 from https://clang.llvm.org/docs/LibTooling.html
[42] LLVM Developers. 2024. LLVM Debian/Ubuntu Nightly Packages. Retrieved October 20, 2024 from https://apt.llvm.org
[43] LLVM Developers. 2024. LLVM Download Page. Retrieved October 20, 2024 from https://releases.llvm.org/
[44] LLVM Developers. 2024. LLVM's Analysis and Transform Passes. Retrieved October 20, 2024 from https://llvm.org/docs/Passes.html
[45] LLVM Developers. 2024. Release 5.X Commits. Retrieved October 20, 2024 from https://github.com/llvm/llvm-project/commits/release/5.x/
[46] Robin Morisset, Pankaj Pawan, and Francesco Zappa Nardelli. 2013. Compiler testing via a theory of sound optimisations in the C11/C++11 memory model. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '13)*. ACM, 187–196. DOI: https://doi.org/10.1145/2491956.2491967
[47] Eriko Nagai, Atsushi Hashimoto, and Nagisa Ishiura. 2014. Reinforcing random testing of arithmetic optimization of C compilers by scaling up size and number of expressions. *Inf. Media Technol.* 9, 4 (2014), 456–465. DOI: https://doi.org/10.11185/IMT.9.456
[48] Yunbo Ni and Shaohua Li. 2025. Interleaving large language models for compiler testing. *Proc. ACM Program. Lang.* 9, OOPSLA2 (Oct. 2025), Article 301, 1–27. DOI: https://doi.org/10.1145/3763079
[49] OpenRouter. 2025. Gemini 2.5 Flash. Retrieved October 15, 2025 from https://openrouter.ai/google/gemini-2.5-flash
[50] OpenRouter. 2025. Qwen Plus 0728. Retrieved October 15, 2025 from https://openrouter.ai/qwen/qwen-plus-2025-07-28
[51] Xianfei Ou, Cong Li, Yanyan Jiang, and Chang Xu. 2024. The mutators reloaded: Fuzzing compilers with large language model generated mutation operators. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 4 (ASPLOS '24)*. ACM, 298–312. DOI: https://doi.org/10.1145/3622781.3674171
[52] John Regehr, Yang Chen, Pascal Cuoq, Eric Eide, Chucky Ellison, and Xuejun Yang. 2012. Test-case reduction for C compiler bugs. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '12)*. ACM, 335–346. DOI: https://doi.org/10.1145/2254064.2254104
[53] Konstantin Serebryany, Derek Bruening, Alexander Potapenko, and Dmitriy Vyukov. 2012. AddressSanitizer: A fast address sanity checker. In *Proceedings of the 2012 USENIX Annual Technical Conference (USENIX ATC '12)*. USENIX Association, 309–318. Retrieved from https://www.usenix.org/conference/atc12/technical-sessions/presentation/serebryany
[54] Chengnian Sun, Vu Le, and Zhendong Su. 2016. Finding compiler bugs via live code mutation. In *Proceedings of the 2016 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA '16)*. ACM, 849–863. DOI: https://doi.org/10.1145/2983990.2984038
[55] Chengnian Sun, Vu Le, Qirun Zhang, and Zhendong Su. 2016. Toward understanding compiler bugs in GCC and LLVM. In *Proceedings of the 25th International Symposium on Software Testing and Analysis (ISSTA '16)*. ACM, 294–305. DOI: https://doi.org/10.1145/2931037.2931074
[56] Theodoros Theodoridis, Manuel Rigger, and Zhendong Su. 2022. Finding missed optimizations through the lens of dead code elimination. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '22)*. ACM, 697–709. DOI: https://doi.org/10.1145/3503222.3507764
[57] Theodoros Theodoridis and Zhendong Su. 2024. Refined input, degraded output: The counterintuitive world of compiler behavior. *Proc. ACM Program. Lang.* 8, PLDI (2024), 671–691. DOI: https://doi.org/10.1145/3656404
[58] Bo Wang, Chong Chen, Junjie Chen, Bowen Xu, Chen Ye, Youfang Lin, Guoliang Dong, and Jun Sun. 2025. A comprehensive study of OOP-related bugs in C++ compilers. *IEEE Trans. Softw. Eng.* 51, 6 (2025), 1762–1782. DOI: https://doi.org/10.1109/TSE.2025.3566490
[59] Bo Wang, Chong Chen, Ming Deng, Junjie Chen, Xing Zhang, Youfang Lin, Dan Hao, and Jun Sun. 2025. Fuzzing C++ compilers via type-driven mutation. *Proc. ACM Program. Lang.* 9, OOPSLA2 (Oct. 2025), Article 316, 1–29. DOI: https://doi.org/10.1145/3763094
[60] Theodore Luo Wang, Yongqiang Tian, Yiwen Dong, Zhenyang Xu, and Chengnian Sun. 2023. Compilation consistency modulo debug information. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '23)*. ACM, 146–158. DOI: https://doi.org/10.1145/3575693.3575740
[61] Jiangchang Wu, Yibiao Yang, Maolin Sun, and Yuming Zhou. 2025. Unveiling compiler faults via attribute-guided compilation space exploration. In *Proceedings of the 2025 USENIX Annual Technical Conference (USENIX ATC '25)*. USENIX Association, 1109–1125. Retrieved from https://www.usenix.org/conference/atc25/presentation/wu-jiangchang
[62] Jiangchang Wu, Yibiao Yang, and Yuming Zhou. 2023. Boosting compiler testing via eliminating test programs with long-execution-time. In *Proceedings of the IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER '23)*. IEEE, 593–603. DOI: https://doi.org/10.1109/SANER56733.2023.00061

[63] Chunqiu Steven Xia, Matteo Paltenghi, Jia Le Tian, Michael Pradel, and Lingming Zhang. 2024. Fuzz4All: Universal fuzzing with large language models. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE '24)*. ACM, Article 126, 1–13. DOI: https://doi.org/10.1145/3597503.3639121

[64] Chenyuan Yang, Yinlin Deng, Runyu Lu, Jiayi Yao, Jiawei Liu, Reyhaneh Jabbarvand, and Lingming Zhang. 2024. WhiteFox: White-box compiler fuzzing empowered by large language models. *Proc. ACM Program. Lang.* 8, OOPSLA2 (2024), 709–735. DOI: https://doi.org/10.1145/3689736

[65] Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. 2011. Finding and understanding bugs in C compilers. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '11)*. ACM, 283–294. DOI: https://doi.org/10.1145/1993498.1993532

[66] Yibiao Yang, Qingyang Li, Maolin Sun, Jing Yang, Jiangchang Wu, and Yuming Zhou. 2025. Isolating compiler faults through differentiated compilation configurations. *IEEE Trans. Softw. Eng.* 51, 6 (2025), 1–16. DOI: https://doi.org/10.1109/TSE.2025.3569530

[67] Yibiao Yang, Maolin Sun, Jiangchang Wu, Qingyang Li, and Yuming Zhou. 2025. Debugger toolchain validation via cross-level debugging. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1 (ASPLOS '25)*. ACM, 280–294. DOI: https://doi.org/10.1145/3669940.3707271

[68] Qirun Zhang, Chengnian Sun, and Zhendong Su. 2017. Skeletal program enumeration for rigorous compiler testing. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '17)*. ACM, 347–361. DOI: https://doi.org/10.1145/3062341.3062379

Received 27 July 2025; revised 9 January 2026; accepted 19 January 2026